

HOOX

END-USER WORKSPACE & CLIENT OPERATIONS MANUAL

Generated: 2026-06-10

Classification: Technical Documentation

Version: 2026-06-10

TABLE OF CONTENTS

ENDUSER

 Hoox Developer Hub

ENDUSER

Installation Guide

ENDUSER

Platform Configuration

ENDUSER

5-Minute Quick Start

ENDUSER

How Hoox Works

ENDUSER

 Edge-First Architecture

ENDUSER

Cloudflare Services Map

ENDUSER

 Idempotency & Durable Objects

ENDUSER

 Signals & Trade Spec

ENDUSER

AI Risk Manager & AI Gateway

ENDUSER

Local Development

ENDUSER

Terminal UI (TUI)

ENDUSER

 Database Operations

ENDUSER

Secrets & Network Security

ENDUSER

Infrastructure Management

ENDUSER

 Deploying to Production

ENDUSER

 Self-Healing & Repair

ENDUSER

TradingView Webhook Integration

ENDUSER

Telegram Bot Setup

ENDUSER

Email Signal Routing

ENDUSER

CLI Commands Reference

ENDUSER

API Endpoint Specification

ENDUSER

Configuration Dictionary



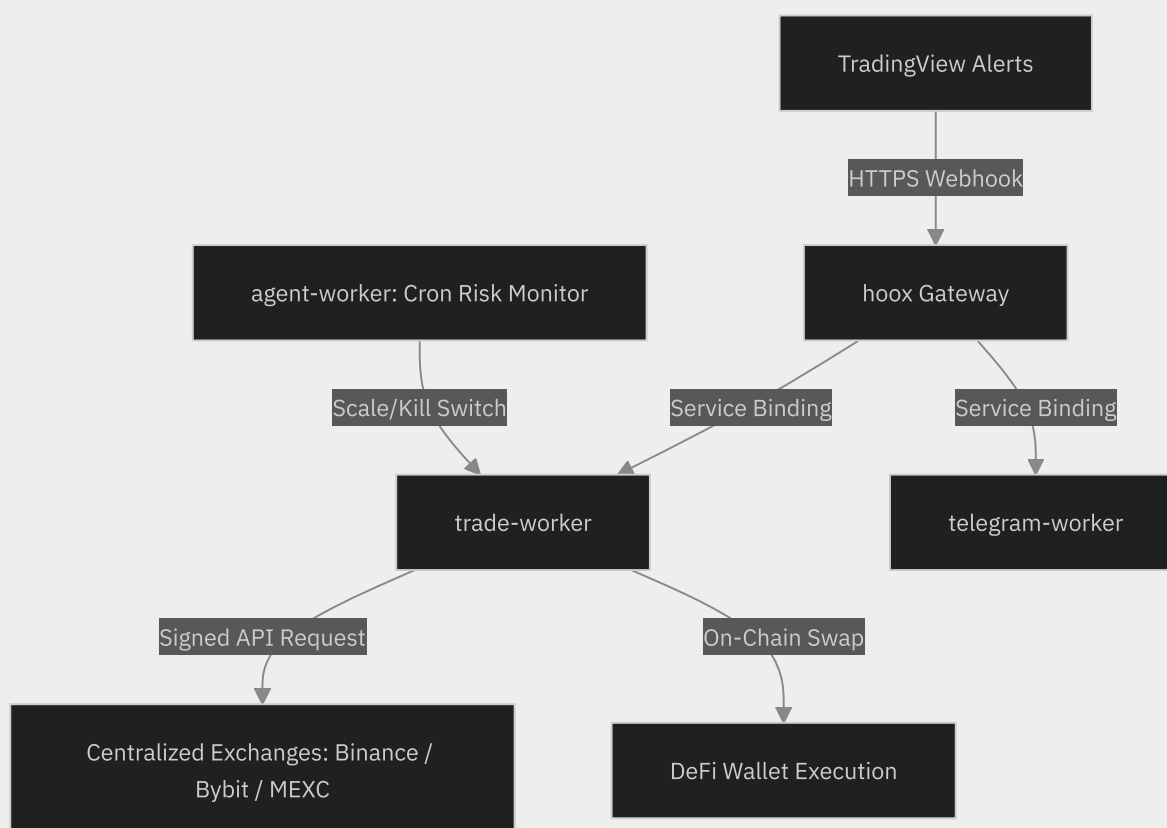
HOOX DEVELOPER HUB

enduser/home.md



WELCOME TO THE HOOX DEVELOPER HUB

*Hoox is a free, open-source, zero-latency algorithmic trading framework and automation engine deployed natively to the **Cloudflare® Edge Network**. By utilizing a distributed microservice architecture, Hoox processes trade signals, evaluates risk parameters, executes order routing, and fires Telegram notifications—all within milliseconds and directly from the edge nodes closest to exchange servers.*



CHOOSE YOUR PATH

Whether you are a retail algorithmic trader setting up your first automated TradingView strategies, a quantitative analyst exploring low-latency DeFi order routing, or a DevOps engineer maintaining multi-exchange infrastructure, our docs are split into highly focused tracks:

1. 🚩 Getting Started

If you are brand new to the Hoox ecosystem, start here to prepare your machine, provision resources, and deploy your first live microservice in under 5 minutes:

- **Core Installation** — Provision prerequisites (Bun runtime, Cloudflare credentials) and bootstrap a project.
- **Platform Configuration** — Declaratively define environment variables, JSON profile templates, and KV keys.
- **5-Minute Quick Start** — Launch local worker runners and execute a simulated webhook trade signal.

2. 🧠 Core Concepts

Understand the underlying technology, low-latency edge architecture, and security layers that protect your api keys:

- **How Hoox Works** — The end-to-end lifecycle of a trade signal from webhook alert to order confirmation.
- **Edge-First Architecture** — Why V8 isolates and Cloudflare Workers outperform traditional VPS architectures by 30-60%.
- **Cloudflare Services Map** — How D1, KV, R2, Queues, Vectorize, and Browser Rendering are integrated.
- **Idempotency & Durable Objects** — Preventing catastrophic double-execution and race conditions during high-frequency events.
- **Signals & Trade Routing** — How parameters map from webhook JSON schemas to live exchange payloads.
- **AI Risk Manager** — The 5-minute autonomous risk scanner, trailing-stop mathematician, and account kill-switch.

3. 🛠️ Operational Guides

Practical runbooks and blueprints for daily operations, local development, and system health maintenance:

- **Local Dev & Workspaces** — Run, hot-reload, and test workers locally via Bun or Docker container stacks.
- **Terminal UI Operations** — Launch and navigate the full-screen 9-view operations cockpit (`./hoox-tui`).
- **Edge Database Operations** — Manage global SQLite schemas, track drizzle migrations, and run D1 queries.
- **Secrets & Network Security** — Secure Cloudflare Zero Trust corridors and encrypt sensitive API credentials.

- **Infrastructure Management** — Spin up and inspect KV, Queues, R2, and Vectorize indexes in one command.
- **Deploying to Production** — Roll out code, bind V8 isolates, and deploy Next.js dashboard consoles.
- **Self-Healing & Repair** — Diagnose connection dropouts, rotate API keys, and run interactive system repair tools.

4. 🎓 Step-by-Step Tutorials

Follow guided, end-to-end tutorials to integrate your trade sources and notification handlers:

- **TradingView Webhook Integration** — Write customized Pine Script v5 indicators that ping the Hoox webhook receiver.
- **Telegram Bot Setup** — Configure real-time order alerts, P&L reports, and secure chat commands.
- **Email Signal Routing** — Configure email parsing services to convert raw inbox alerts into edge trade executions.

5. 📖 Reference Material

Deep specifications, schema registries, and command-line manual trees:

- **CLI Commands Index** — Complete options, positional arguments, and JSON flag trees for the `hoox` tool.
- **API Spec & REST Routes** — HTTP endpoints, request payloads, response templates, and edge errors list.
- **Configuration Properties** — Comprehensive dictionary of all 31 env keys and 16 KV key-value items.



OFFLINE REFERENCE & AI CONTEXT

Download our consolidated specifications or view complete text packages for feeding into LLMs:

- **Download Enduser Full PDF Manual** — Complete concatenated offline guide.
- **Download DevOps Full PDF Manual** — Complete concatenated offline DevOps spec.
- **View Consolidated LLM Context Text** — Giant single-file text format for AI/LLM models.

💡 Tip

Looking for deep engineering plans, DDL SQL schemas, or CI/CD deployment workflows? Check out the **DevOps & Architecture Manual** for developer-centric operator blueprints!

INSTALLATION GUIDE

enduser/getting-started/installation.md



INSTALLATION GUIDE

This guide walks you through preparing your environment, installing the `@jango-blockchained/hoox-cli` tool, cloning the microservices monorepo recursively, and validating your local system prerequisites.



SYSTEM PREREQUISITES

Before installing the Hoox command-line workspace, ensure your system meets the following standard software requirements:

1. 🍌 Bun JavaScript Runtime (Version ≥ 1.2)

Hoox uses **Bun** as its primary package manager, script runner, and native testing engine. Bun's blazing-fast startup time and zero-config TypeScript compilation are critical to our local developer feedback loops.

- macOS / Linux:

```
curl -fsSL https://bun.sh | bash
```

- Windows (via Powershell):

```
powershell -c "irm https://bun.sh/install.ps1 | iex"
```

2. ⚡ Cloudflare Account & Wrangler CLI

All Hoox workers are compiled to run on **Cloudflare's Edge V8 isolates**. You will need a free Cloudflare account:

- **Account:** [Register a free account](#) (the free tier provides 100,000 requests/day, D1 database, KV storage, Queues, R2, and vector search—costing \$0/month).
- **Cloudflare CLI:** Ensure you have `wrangler` installed globally (though Hoox manages local wrangler operations automatically, having it indexed globally is recommended):

```
bun add -g wrangler
```

3. 🐳 Docker & Docker Compose (Optional)

If you prefer running the entire Hoox local trading workspace in fully isolated container environments with one command, ensure you have **Docker Desktop** installed and running:

- Check status: `docker compose version`



OPTION A: INSTALL VIA PACKAGE MANAGER (RECOMMENDED)

To install the global management CLI tool directly from npm/bun registry to manage new workspaces:

```
# Install globally using Bun
bun add -g @jango-blockchained/hoox-cli
```

Once installed, verify that the `hoox` command is registered globally in your system path:

```
hoox --version
```



OPTION B: BUILD & RUN FROM SOURCE

If you plan to contribute to the Hoox CLI packages or prefer working directly inside a monolithic source clone:

```
# 1. Clone the parent repository with git submodules recursively
git clone --recursive https://github.com/jango-blockchained/hoox-setup.git hoox-trading
cd hoox-trading

# 2. Install all monorepo workspace dependencies via Bun
bun install

# 3. Verify the locally linked CLI runs from root
bun run build:cli
./packages/cli/bin/hoox.js --help
```

⚠ Warning

You MUST use `git clone --recursive` or run `git submodule update --init --recursive` after cloning. If you omit submodules, the worker directories under `workers/` will be empty and deployments will fail.



BOOTSTRAPPING YOUR TRADING WORKSPACE

Now, initialize your new algorithmic trading workspace. This compiles directories, configures workspace settings, and sets up Git tracking.

```
# Bootstrap your trading folder
hoox clone my-trading-empire
cd my-trading-empire
```

This command automatically structures your directories as a clean monorepo:

```
my-trading-empire/
├── packages/
│   ├── cli/      # Workspace CLI management binaries
│   ├── tui/      # 9-view Terminal UI monitoring cockpit
│   └── shared/    # Reusable routers, rate-limiters, & errors
├── workers/
│   ├── hoox/     # Gateway, rate-limiter, DO idempotency lock
│   ├── trade-worker/ # Multi-exchange execution engine
│   └── ...        # Other analytical and web3 wallet workers
└── package.json
```



RUNNING THE INTERACTIVE SETUP WIZARD

With your folder structured, execute the interactive bootstrap wizard:

```
hoox init
```

The CLI wizard will guide you through 4 critical setup phases:

1. **Toolchain Validation:** Probes your machine for `bun`, `git`, `wrangler`, and `docker` configurations.
2. **Cloudflare Authentication:** Prompts for your Cloudflare API token (with `Account.D1`, `Account.KV`, `Account.Workers` permissions) and Account ID.
3. **Microservice Profile Selection:** Lets you declaratively toggle which edge workers to enable (e.g., enable Gateway and Trade Worker, disable Web3 Wallet if you only trade centralized exchanges).
4. **Local Credentials Encryption:** Generates safe local `.dev.vars` matrices and sets up initial KV configuration structures.

hoox Setup & Initialization

```
✓ bun (v1.2.1) found
✓ wrangler (v3.50.0) found
✓ git found
✓ Cloudflare Credentials Verified

Enable central Gateway Worker? [Y/n]: y
Enable Multi-Exchange trade-worker? [Y/n]: y
Enable agent-worker AI Risk Manager? [Y/n]: y
```

VERIFYING LOCAL PREREQUISITES

Verify that all dependencies and Cloudflare edge routes are accessible:

```
# Perform detailed pre-flight diagnostic checklist
hoox check prerequisites
```

If all diagnostic checks pass, you are ready to proceed with configuration!

Tip

Got installation issues? Run `hoox repair check` to automatically analyze common path resolution issues, missing environment variables, or node-gyp build failures, and recover your local workspace seamlessly.

Next Steps

- **Configuration Matrix** — Map out your 31-key environment variables and 16-key KV registries.
- **5-Minute Quick Start Guide** — Execute your first simulated edge trade in under 5 minutes.

PLATFORM CONFIGURATION

enduser/getting-started/configuration.md



PLATFORM CONFIGURATION

Hoox uses a dual-layer configuration topology: a **declarative build-time environment layer** (managed via local files and Cloudflare Secrets) and an **instantaneous runtime configuration layer** (managed via Cloudflare KV key-value databases). This ensures maximum agility: deploy secure code once, and adjust trading parameters or flip kill switches in real-time without redeploying code.

1. DECLARATIVE ENVIRONMENT VARIABLES (`.env.local`)

For local operations, build-time variables, and workspace linking, Hoox loads a `.env.local` file at your project root.

Copy the secure template during initialization:

```
cp .env.example .env.local
```

The Hoox environment is split into 5 core logical sections. Below is the comprehensive dictionary of key configuration parameters:

A. ⚡ Cloudflare Core Infrastructure

Parameter	Required	Description	Example
<code>CLOUDFLARE_API_TOKEN</code>	Yes	API token with <code>D1</code> , <code>KV</code> , <code>Queue</code> , and <code>Workers</code> write permissions.	<code>cf_pat_abc123...</code>
<code>CLOUDFLARE_ACCOUNT_ID</code>	Yes	Your unique 32-character Cloudflare dashboard account hash.	<code>debc6545e63bea36be059cbc82d80ec8</code>
<code>SUBDOMAIN_PREFIX</code>	Yes	Subdomain prefix under which your public gateway routes compile.	<code>hoox-edge</code>

B. 💰🔑 Exchange API Keys (Securely Redacted)

These credentials must be injected as secure encrypted environment variables into your Cloudflare Worker isolates. The Hoox CLI automates this injection.

- **Binance:**
 - `BINANCE_API_KEY` — Your read/write exchange account trade permission key.
 - `BINANCE_API_SECRET` — Private secret key used to HMAC-SHA256 sign order payloads.
- **Bybit:**
 - `BYBIT_API_KEY` — Order routing account key.
 - `BYBIT_API_SECRET` — Order signing private key.
- **MEXC:**
 - `MEXC_API_KEY` — Order routing account key.
 - `MEXC_API_SECRET` — Order signing private key.

C. Telegram Bot Alerts

Parameter	Required	Description	Example
<code>TELEGRAM_BOT_TOKEN</code>	No	HTTP API authentication token provided by Telegram's BotFather.	<code>123456789:ABCdefGh...</code>
<code>TELEGRAM_CHAT_ID</code>	No	The numeric chat ID of the user, group, or channel where alerts route.	<code>987654321</code>

D. Multi-Provider AI Credentials

Used to power autonomous risk assessments, Telegram conversation loops, and time-series summaries in `agent-worker`.

- `OPENAI_API_KEY` — Access key for OpenAI GPT models.
- `ANTHROPIC_API_KEY` — Access key for Anthropic Claude models.
- `GOOGLE_AI_API_KEY` — Access key for Google Gemini models.

2. MANAGING ENVIRONMENT & SECRETS VIA CLI

To prevent plain-text exposure and ensure proper edge variable binding, **never** manually paste sensitive keys into `wrangler.jsonc` files. Instead, utilize the high-integrity `hoox config env` command groups:

```
# 1. Start the guided terminal config wizard
hoox config env init

# 2. View active workspace configuration (sensitive credentials automatically redacted)
hoox config env show

# 3. Validate env file structure against required platform variables
hoox config env validate

# 4. Generate local decrypted .dev.vars files for every enabled worker
hoox config env generate-dev-vars
```

Note

Decrypted `.dev.vars` files contain local environment credentials and are excluded from git history via `.gitignore` automatically. Running `hoox config env generate-dev-vars` ensures that local worker testing via `bun run dev` has access to simulated credentials safely.

3. WORKER CONFIGURATIONS (`wrangler.jsonc`)

Each worker in the monorepo has a standard `wrangler.jsonc` file at its directory root. These configurations declare:

1. **Service Bindings:** Connects gateway routes (`hoox`) to internal compute units (`trade-worker` , `d1-worker`) directly via microsecond V8 isolates—no TCP/TLS overhead, no public routes.
2. **Resource Bindings:** Declares which D1 databases, R2 storage buckets, and KV configurations are linked.
3. **Execution Mode:** Toggles latency-saving features like **Smart Placement** (`"placement": { "mode": "smart" }`).

You can inspect, check, or change worker toggle status directly:

```
# Print complete microservice enabled/disabled status tree
hoox config show

# Declaratively enable/disable specific workers in the manifest
hoox config set workers.web3-wallet-worker.enabled false
```

4. RUNTIME KV CONFIGURATION (SUB-MILLISECOND SETTINGS)

One of Hoox's core architectural features is **instant runtime parameter updates**. By using Cloudflare KV, certain settings can be modified globally in sub-milliseconds and take effect on the very next signal execution—without rebuilding or redeploying any worker.

Hoox manages a standard **16-key runtime manifest** inside the `CONFIG_KV` namespace. Below are the most critical runtime parameters:

KV Key	Type	Default	Description
<code>trade:kill_switch</code>	<code>boolean</code>	<code>false</code>	Global Kill Switch. If set to <code>true</code> , all incoming trade execution loops immediately halt.
<code>trade:max_daily_drawdown_percent</code>	<code>number</code>	<code>5</code>	Maximum daily drawdown before the AI Risk Manager flips the kill switch.
<code>webhooks:queue_mode</code>	<code>string</code>	<code>queue_failover</code>	Trade delivery behavior: <code>direct_only</code> , <code>queue_failover</code> , or <code>queue_everywhere</code> .
<code>exchanges:default_routing</code>	<code>string</code>	<code>bybit</code>	Default centralized exchange router. Options: <code>binance</code> , <code>bybit</code> , <code>mexc</code> .

Managing KV Settings via CLI

```
# Get the current value of the Global Kill Switch
hoox config kv get trade:kill_switch

# Manually trigger a global trading halt
hoox config kv set trade:kill_switch true

# Restore trading by flipping the switch back
hoox config kv set trade:kill_switch false

# Declaratively apply default manifest variables to Cloudflare KV
hoox config kv apply-manifest
```

Tip

Changed exchange configurations or toggled the kill switch? There is **no need** to redeploy. The changes are distributed across Cloudflare's 330+ global edge locations near-instantaneously!

Next Steps

- **5-Minute Quick Start Guide** — Launch local worker runners and execute a simulated webhook trade signal.

- **Deploying to Production** — Deploy and bind all workers in the correct dependency sequence.

5-MINUTE QUICK START

enduser/getting-started/quick-start.md



5-MINUTE QUICK START

This guide gets you from a blank console to a **fully active, edge-deployed algorithmic trading ecosystem** on the Cloudflare network, processing simulated signals in under 5 minutes.



STEP-BY-STEP DEPLOYMENT PATH

Step 1: Initialize Workspace Credentials

If you haven't already run the setup wizard during installation, execute the initial workspace configuration:

```
hoox init
```

Follow the interactive prompts to paste your Cloudflare Account ID and API Token, and define your unique `SUBDOMAIN_PREFIX` (e.g., `alpha-trading`).

Step 2: Inject Encrypted Exchange API Keys

For your safety, exchange credentials (API keys and private signatures) are never stored in plain text. Inject them as encrypted **Cloudflare Workers Secrets** bound securely to your compute instances:

```
# Inject Bybit Credentials
hoox secrets set BYBIT_API_KEY "your_bybit_key_here"
hoox secrets set BYBIT_API_SECRET "your_bybit_secret_here"

# Optional: Inject Binance Credentials
hoox secrets set BINANCE_API_KEY "your_binance_key_here"
hoox secrets set BINANCE_API_SECRET "your_binance_secret_here"
```

⚠ Warning

Cloudflare Secrets are encrypted at rest using hardware-level keys and are injected straight into your V8 execution isolates at runtime. They can never be decrypted or read back via the API, ensuring top-tier security for your capital.

Step 3: Deploy All Workers in Sequence

Hoox microservices communicate internally via Service Bindings. The CLI automatically manages the deployment sequence, ensuring databases, queues, and configuration stores compile first, followed by gateway routers and background compute tasks:

```
# Compile and deploy all enabled workers
hoox deploy all --auto
```

This command automatically provisions:

1. **D1 Edge Database** (`hoox-db`)
2. **CONFIG_KV** configuration namespace
3. **Internal Workers** (`trade-worker` , `d1-worker` , `telegram-worker`)
4. **Public Gateway** (`hoox` gateway router)
5. **Next.js Dashboard Command Center** (`workers/dashboard`)

Once completed, the CLI will output your public Gateway endpoint URL: `https://hoox.alpha-trading.workers.dev`

Step 4: Fire a Simulated Trade Webhook

Now, fire a test webhook trade signal to your live gateway using `curl` .

```
curl -X POST https://hoox.alpha-trading.workers.dev/webhook \
-H "Content-Type: application/json" \
-d '{
  "apiKey": "your-hoox-webhook-passkey",
  "exchange": "bybit",
  "action": "LONG",
  "symbol": "BTCUSDT",
  "quantity": 0.001,
  "leverage": 10
}'
```



WEBHOOK PAYLOAD PARAMETERS SPEC

Every webhook payload fired to your Gateway must match the following JSON Schema:

Parameter	Type	Required	Description
<code>apiKey</code>	<code>string</code>	Yes	Your custom webhook authorization passkey (defined in <code>CONFIG_KV</code>).
<code>exchange</code>	<code>string</code>	Yes	Target exchange router: <code>binance</code> , <code>bybit</code> , or <code>mexc</code> .
<code>action</code>	<code>string</code>	Yes	Trading action direction: <code>LONG</code> (buy/open), <code>SHORT</code> (sell/open), or <code>CLOSE</code> (flatten position).
<code>symbol</code>	<code>string</code>	Yes	Standard market symbol.
<code>quantity</code>	<code>number</code>	Yes	Position size / quantity.
<code>leverage</code>	<code>number</code>	No	Leverage coefficient. Defaults to <code>1</code> (spot) if omitted.



EXPECTED SUCCESS RESPONSE

When a signal arrives, the Hoox Gateway authorizes the request, locks execution via Durable Objects, routes order calculations to the edge node nearest to Bybit's servers, executes the order, and registers the transaction in your D1 SQLite table.

You will receive an instantaneous, low-latency JSON response:

```
{
  "success": true,
  "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "exchange": "bybit",
  "symbol": "BTCUSDT",
  "action": "LONG",
  "result": {
    "orderId": "18049284739",
    "status": "Filled",
    "executedQty": 0.001,
    "price": 68425.5,
    "timestamp": 1779261050000
  }
}
```



Tip

If the exchange is temporarily undergoing system maintenance or experiences high network congestion, Hoox will automatically intercept the failure, enqueue the trade in **Cloudflare Queues** with exponential backoff retry policies, and return a `"status": "Enqueued"` response to guarantee delivery!



Next Steps

- **How Hoox Works** — Take a deep dive into the edge processing pipelines.
- **Terminal UI Guide** — Run, hot-reload, and inspect live metrics in a full-screen console interface.
- **Set Up Telegram Alerts** — Link your bot to get order fills pushed straight to your phone.

HOW HOOX WORKS

enduser/concepts/how-hoox-works.md

HOW HOOX WORKS

At its core, **Hoox** functions as a highly resilient, low-latency pipeline that intercepts trading signals from external sources, validates and decodes the payloads, and converts them into cryptographically signed order placements on global exchanges in milliseconds.

Here is the exact lifecycle of an edge trade execution, from alert trigger to brokerage fill.

THE EXECUTION PIPELINE



DETAILED PIPELINE PHASE ANALYSIS

1. Signal Arrival & Ingestion

Signals represent specific instructions to buy, sell, or close positions. Hoox ingests these instructions through three primary channels:

- **TradingView Webhooks:** High-integrity trade alerts configured via Pine Script that issue JSON POST payloads to your gateway's public `/webhook` route.
- **Telegram Commands:** Direct user commands sent to your private Telegram Bot (e.g. `/trade buy btc quantity=0.01 exchange=bybit`), parsed via your webhook router.

- **Email Signals:** Secure email triggers forwarded from specialized alerts (e.g. TradingView email alerts), parsed natively by `email-worker`.

2. Edge Firewall & Gateway Validation

When a signal hits your public endpoint, the `hoox gateway` interceptor immediately evaluates the request inside a sandboxed V8 isolate:

1. **API Key Authorization:** Authenticates the payload's `apiKey` property against your secure manifest stored in Cloudflare KV.
2. **IP Allow-listing:** Verifies that the client IP matches a authorized IP range in KV (essential for restricting access solely to TradingView's known webhook IPs).
3. **Global Kill Switch check:** Ensures `trade:kill_switch` is `false`. If `true`, the pipeline aborts immediately to protect your capital.
4. **Rate Limiting:** Verifies that signal frequency does not exceed safe execution thresholds (default: 10 orders/minute).
5. **Idempotency Check:** Locks a unique transaction trace ID using a **SQLite-backed Durable Object**. If the DO detects that the exact same signal hash was already processed, it drops the request to prevent double-ordering.

3. Service Binding Order Routing

Once validated, the gateway bypasses public internet routing and calls the internal `trade-worker` directly via **Service Bindings**:

- **Zero Latency:** Communication occurs within a microsecond inside the V8 engine, with zero TCP handshakes or TLS decryption overhead.
- **Private Isolation:** The `trade-worker` does not expose any public URL, remaining completely isolated from external attacks.
- **Dynamic Exchange Routing:** The worker parses the symbol (e.g. `BTCUSDT`) and evaluates your KV config. If Bybit is undergoing maintenance, you can instantly redirect trades to MEXC or Binance with one command.

4. Exchange Execution & Cryptographic Signing

The `trade-worker` loads your encrypted API credentials from Cloudflare Secrets, computes the HMAC-SHA256 signature for the order parameters, and pushes the signed request to the exchange's nearest API gateway:

- **Smart Placement:** Because Smart Placement is enabled, Cloudflare automatically runs your worker on the physical edge node located geographically closest to the exchange's servers (e.g.

Frankfurt for Bybit, Tokyo for Binance), cutting network transit time by up to 60%.

5. Persistent D1 Logging & Telemetry

Upon receipt of the order response (e.g. `Filled`, `Partial`, or `Rejected`), Hoox updates your ledger:

- **D1 SQLite Database:** Writes transaction logs, filled prices, execution fees, and order IDs to your D1 database.
- **R2 Log Offloading:** Offloads full JSON request-response packets to your secure R2 storage bucket to keep D1 database footprints compact.

6. User Notifications & Alerts

Finally, the `trade-worker` pings `telegram-worker` via an internal binding:

- You receive an immediate Telegram notification on your phone detailing the symbol, filled price, order status, and transaction ID.
- Twice-daily, `report-worker` uses Browser Rendering to compile beautiful HTML reports of your trading metrics, generating PDFs and dropping the link directly in your chat.

Tip

If the exchange API experiences transient network dropouts or severe rate limits, the Hoox Gateway automatically transfers the payload to **Cloudflare Queues** with an exponential retry schedule (`30s` → `1m` → `5m` → `15m`), keeping your strategy fully reliable even during periods of heavy market volatility.

Next Steps

- **Edge-First Architecture** — Understand the absolute latency benefits of edge workers vs VPS.
- **Signals & Trade Specifications** — Analyze the complete JSON webhook and order formatting.



EDGE-FIRST ARCHITECTURE

enduser/concepts/edge-architecture.md



EDGE-FIRST ARCHITECTURE

Why running algorithmic trading bots on V8 isolates and Cloudflare's 330+ global data centers cuts latency by 60% and eliminates slippage.



THE PHYSICS OF LATENCY: WHY TRADITIONAL VPS BOTS FAIL

Traditional trading bots are deployed on a single virtual private server (VPS) in a fixed geographical data center (e.g., Virginia, USA).

The VPS Bottleneck (200ms+ slippage)

1. **Signal Generation:** A TradingView alert fires in London.
2. **Network Transit:** The alert travels across the Atlantic to your Virginia VPS (~85ms).
3. **Execution Delay:** The VPS boots a heavy Node/Docker runtime to process the signal (~10ms).
4. **Exchange Transit:** The order travels from Virginia to Bybit's API servers in Tokyo or Frankfurt (~110ms).
5. **Total Transit Latency:** **205ms** before the exchange matches your order.

```
[London Alert] — (85ms) —> [Virginia VPS] — (110ms) —> [Frankfurt Bybit] = 205ms Latency
```

The Hoox Edge Path (Under 15ms latency)

1. **Signal Generation:** A TradingView alert fires in London.
2. **Edge Ingestion:** The alert hits Cloudflare's nearest London Point of Presence (PoP) in **1.8ms**.
3. **V8 Isolate Processing:** A sandboxed V8 isolate processes and validates the order instantly (<3ms).
4. **Smart Placement Routing:** The internal service binding transfers compute to the edge node geographically closest to Bybit's servers (Frankfurt) within **8ms**.
5. **Total Edge Latency:** **Under 15ms** total network transit time.

```
[London Alert] — (1.8ms) —> [London PoP] — (8ms Service Binding) —> [Frankfurt Node] — (1ms)
```




V8 ISOLATES VS. TRADITIONAL VMS / CONTAINERS

Traditional architectures run on Virtual Machines or Docker containers. These runtimes carry significant memory overhead and introduce **cold starts**—the time required to spin up a container after inactivity.

Architectural Dimension	Traditional VM / Container (VPS)	Cloudflare Edge Worker (V8 Isolate)
Boot Architecture	Heavy guest OS, Node runtime, npm modules	Sandboxed JavaScript V8 Engine runtime
Cold-Start Delay	1,000ms – 15,000ms (system stalls)	< 3ms (virtually non-existent)
Memory Footprint	256MB – 2GB per instance	128MB max (highly lightweight)
Global Failover	Requires complex DNS/Load Balancer setup	Natively distributed across 330+ locations
Geographic Location	Fixed data center	Automatically routes to nearest user node
Scaling Model	Manual provisioning or autoscaling groups	Automatic — each request may hit a different PoP
DDoS Protection	Requires separate service (e.g. Cloudflare)	Built-in at the network edge
Cost Model	Pay for idle capacity 24/7	Pay-per-request; zero cost when idle



SMART PLACEMENT: ZERO-CONFIG LATENCY OPTIMIZER

To achieve sub-millisecond edge execution, Hoox enables Cloudflare's **Smart Placement** engine natively on all critical workers:

```
"placement": {  
  "mode": "smart"  
}
```

How Smart Placement Works

1. When you deploy `trade-worker`, Cloudflare monitors its network dependencies. It notices that `trade-worker` makes outbound signed HTTPS REST requests to Bybit's Frankfurt server (`api.bybit.com`) and writes transaction rows to the `d1-worker` SQLite database.
2. Instead of running the worker's CPU compute at the location where the user's browser or alert hits (e.g. California), Smart Placement **automatically shifts the compute isolate** to the Cloudflare node physically closest to the Bybit servers (Frankfurt).

3. The V8 engine performs all signature calculations and database writes at the edge gateway node, reducing the final API transit time to **under 2 milliseconds**.

Latency Comparison: Real Numbers

Scenario	VPS (Virginia)	Edge (Smart Placement)	Improvement
London → Exchange (Bybit/DE)	~205ms	~10.8ms	19x faster
New York → Exchange (Binance/US)	~45ms	~8ms	5.6x faster
Tokyo → Exchange (Binance/JP)	~120ms	~3ms	40x faster
Singapore → Exchange (MEXC/SG)	~85ms	~5ms	17x faster



Note

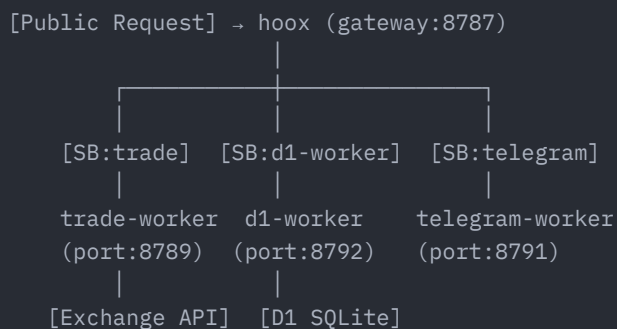
Actual latency depends on exchange API server location, network conditions, and Cloudflare PoP availability. Measurements represent typical round-trip times observed in production.

 HARDWARE-LEVEL SECURITY

Because V8 isolates run in strictly isolated memory contexts managed directly by Google’s Chromium V8 engine, your code is secure:

- **Isolate Protection:** Memory leaks, side-channel attacks, and adjacent tenant exploits are prevented at the V8 compiler level. Each tenant’s code runs in a completely separate memory space with zero shared state.
- **Microsecond Service Bindings:** Communication between workers is processed entirely inside the local V8 runtime, meaning sensitive exchange payloads and API calls never travel over the public internet. This eliminates TLS decryption overhead and packet-sniffing risks.
- **No Shared Filesystem:** Unlike container-based approaches, V8 isolates have no persistent filesystem access — preventing supply-chain attacks via compromised dependencies writing to disk.

Service Binding Architecture



All **SB:** prefixed connections occur **inside the V8 engine** with zero TCP handshakes. External connections (Exchange APIs, Telegram Bot API) use standard HTTPS egress.

Tip

By removing VPS management, you do not just save latency—you also save operational overhead. Cloudflare natively manages automatic scaling, load balancing, SSL negotiation, and route failovers for free.

Next Steps

- **Cloudflare Services Explained** — Learn how D1 SQLite databases, KV, and Queues fit together on the edge.
- **AI Risk Manager** — Understand how autonomous risk checks run on global cron schedules.

CLOUDFLARE SERVICES MAP

enduser/concepts/cloudflare-services.md



CLOUDFLARE SERVICES MAP

Hoox is built natively on a **serverless microservice architecture** powered by a suite of fully integrated Cloudflare® services. By offloading database scaling, messaging retry loops, file storage, and AI inference to Cloudflare's global infrastructure, Hoox runs with **zero server maintenance, ultra-low latency, and \$0 monthly costs** on the free tier.

Here is an architectural map of how each service functions in the Hoox monorepo.



1. D1 DATABASE (EDGE SQL ENGINE)

- **Concept:** A serverless, globally distributed SQLite database engine natively hosted at Cloudflare's edge locations.
- **Hoox Implementation:** Used to store critical transactional data, including executed trades ledger, historical win rates, daily asset balances, and position tracking tables.
- **Why it Matters:**
 - Writes are atomic and ACID-compliant.
 - Queries complete in under 5 milliseconds because the database runs in the same V8 isolate context as your worker logic.
 - No need to host, secure, patch, or configure connection pools for a separate database server (like PostgreSQL or MySQL).



2. KV STORE (SUB-MILLISECOND KEY-VALUE CONFIGURATION)

- **Concept:** A highly available, globally distributed key-value store optimized for high-read/low-write operations.
- **Hoox Implementation:** Houses the global 16-key runtime manifest, including the **Global Kill Switch** (`trade:kill_switch`), exchange routing paths, rate-limiter profiles, and authorized client IP ranges.
- **Why it Matters:**
 - Read operations are cached directly at Cloudflare's edge nodes, delivering sub-millisecond lookups.
 - Updates propagate worldwide in under 10 seconds. You can toggle settings in your terminal, and the change affects every worker globally in seconds, without code redeployment.

3. CLOUDFLARE QUEUES (ASYNCHRONOUS MESSAGING GUARD)

- **Concept:** A high-integrity, serverless message broker guaranteeing at-least-once delivery with built-in retry schedules.
- **Hoox Implementation:** Acts as an asynchronous buffer and retry buffer for signal executions (`queue_failover` mode).
- **Why it Matters:**
 - If a centralized exchange (like Bybit or Binance) undergoes system maintenance or experiences rate limits, the Hoox Gateway intercepts the API error, serializes the trade payload, and enqueues it.
 - The queue automatically retries execution with an exponential backoff schedule (`30s` → `1m` → `5m` → `15m` → `30m`). Your trades never get lost due to internet dropouts or API errors.

4. DURABLE OBJECTS (DISTRIBUTED COORDINATION & IDEMPOTENCY)

- **Concept:** Single-threaded, in-memory compute isolates containing persistent local storage, ensuring that exactly one instance of an object runs globally.
- **Hoox Implementation:** Drives the Gateway's **Idempotency Engine** inside `workers/hoox` .
- **Why it Matters:**
 - When TradingView fires multiple retries for the same alert (due to a transient network timeout), Durable Objects intercept the request, acquire an atomic mutex lock, and evaluate the trade's unique hash.
 - Prevents disastrous duplicate trades (e.g. accidentally buying the same spot position twice) during moments of high market congestion.

5. R2 OBJECT STORAGE (ZERO-EGRESS ASSET VAULT)

- **Concept:** A fully S3-compatible, serverless object storage bucket featuring **zero bandwidth egress fees**.
- **Hoox Implementation:** Stores large data payloads, verbose exchange API request/response packets, and compiled HTML/PDF reports.
- **Why it Matters:**
 - Offloading heavy system logging to R2 saves massive write capacity on your D1 SQL database, keeping your database compact and responsive.
 - Zero-egress pricing means you can retrieve, export, and audit your historic trade logs as many times as you like without incurring network charges.



6. VECTORIZE & WORKERS AI (EMBEDDED RAG & LLMS)

- **Concept:** A serverless vector search database and an on-demand edge AI inference engine running specialized open-source models (like LLaMA 3, Qwen, and Mistral).
- **Hoox Implementation:** Integrates with the Telegram Bot (`telegram-worker`) to provide semantic trade analysis and context-aware natural language conversations.
- **Why it Matters:**
 - User chat queries are converted into high-dimensional vector embeddings, searched against historical trades inside `Vectorize` , and fed into `Workers AI` as context.
 - Deliver highly intelligent, context-aware risk analysis directly on your mobile chat without calling expensive external APIs.



7. BROWSER RENDERING (EDGE PUPPETEER RENDERER)

- **Concept:** Serverless Chrome instances running at Cloudflare's edge, allowing developers to automate browser actions, interact with websites, and convert web elements to documents.
- **Hoox Implementation:** Deployed in `report-worker` to generate twice-daily, styled PDF portfolio reports.
- **Why it Matters:**
 - Reads D1 database P&L records, renders a beautiful, secure HTML5 dashboard report, captures it via Puppeteer, converts it to PDF, saves the file to R2, and links it directly in your Telegram alert.
 - Completely automated, running entirely in the background near-instantaneously.

Tip

Every single one of these services is supported by the Hoox CLI! You can check database tables using `hoox db query` , provision R2 buckets with `hoox infra r2 create` , and sync your KV manifest using `hoox config kv apply-manifest` instantly.



Next Steps

- **Idempotency Mechanics** — Dive into the V8 Durable Objects coordination engine.
- **AI Risk Manager** — Understand how autonomous edge cron jobs evaluate drawdowns and manage trailing stops.



IDEMPOTENCY & DURABLE OBJECTS

enduser/concepts/idempotency.md



IDEMPOTENCY & DURABLE OBJECTS

How Hoox utilizes Cloudflare Durable Objects to guarantee exactly-once execution and prevent catastrophic duplicate orders during network dropouts.



THE DANGER: HOW WEBHOOK RETRIES LEAD TO DOUBLE-ORDERING

Without idempotency, a typical signal failure sequence looks like this:

```
[TradingView Webhook] — (Signal Post) —> [Gateway Node] — (Submit Order) —> [Exchange API]
                                                                    |
                                                                    (Order Filled)
                                                                    |
[TradingView Webhook] <— (TLS/TCP Dropout) — [Gateway Node] <— (Send Success) — (Connection Closed)
|
(No response: Retries!)
|
[TradingView Webhook] — (Signal Post) —> [Gateway Node] — (Submit Order) —> [Exchange API]
                                                                    |
                                                                    (DOUBLE-FILL)
```

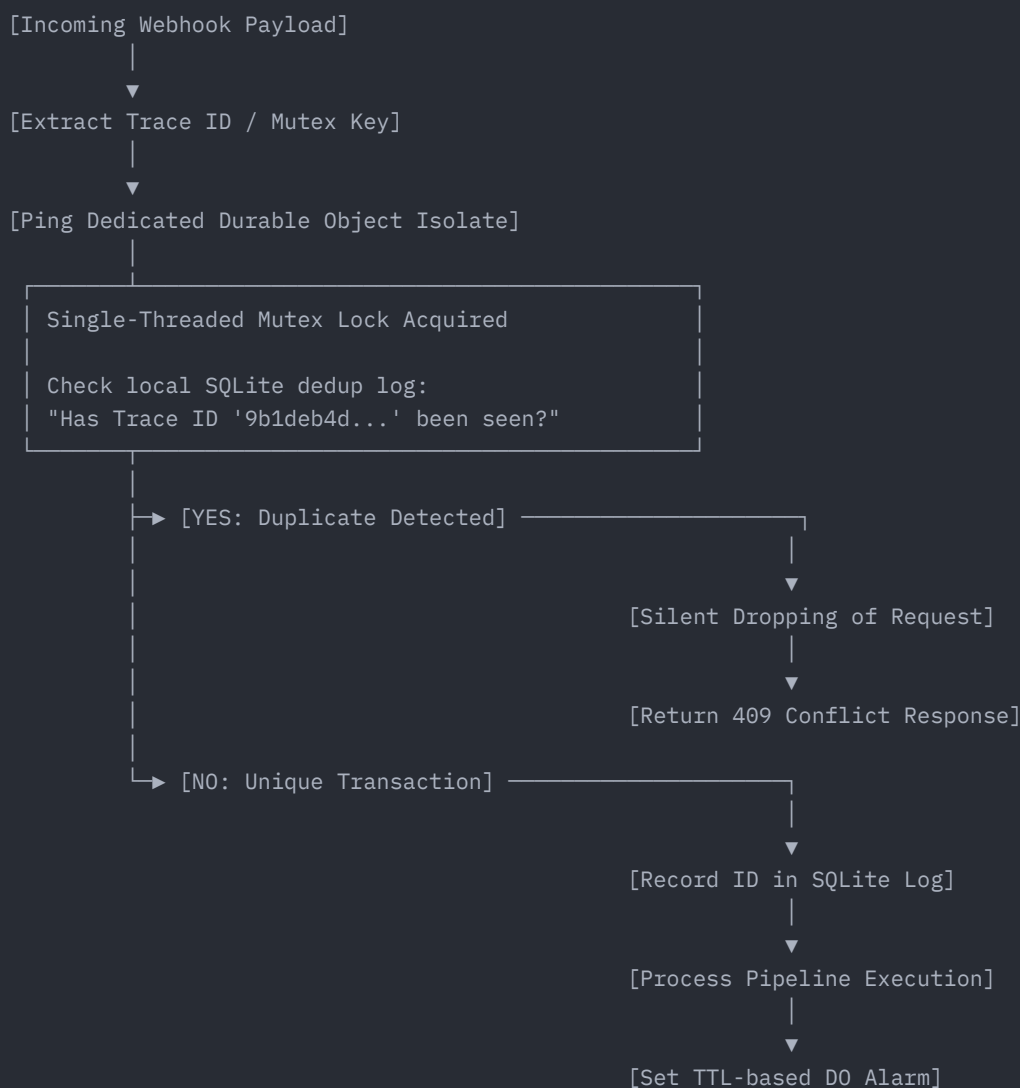


THE HOOX SOLUTION: DURABLE OBJECTS MUTEX LOCKING

To enforce exactly-once execution, Hoox implements an **atomic dedup lock** inside `workers/hoox` utilizing **Cloudflare Durable Objects**.

A Durable Object is a unique, single-threaded compute isolate managed by Cloudflare that maintains its own highly optimized, in-memory state and persistent on-disk SQLite storage. Because access to a specific Durable Object instance is single-threaded, it acts as an absolute **distributed lock** (mutex).

The Idempotency Workflow



THE DEDUP & CLEANUP ALGORITHM

1. Trace ID Generation

When a trade signal is fired, it must include a unique transaction signature.

- For TradingView webhooks, this is automatically generated as a combination of alert parameters and timestamps.
- If a client does not supply a transaction ID, the Hoox Gateway dynamically hashes the symbol, exchange, action, and timestamp to create a unique **Idempotency Key**.

2. Atomic Evaluation

Before executing any order routing:

1. The gateway routes the request to the namespace-mapped Durable Object using the transaction ID as the binding key.
2. The Durable Object checks its internal state. Since DOs are single-threaded, there is **zero risk of race conditions**—if two identical HTTP requests hit Cloudflare simultaneously, they are processed sequentially inside the DO.
3. If the ID exists in the DO's SQLite log, the DO immediately intercepts the request and throws a `409 Conflict` exception, stopping the pipeline before hitting exchange APIs.
4. If unique, it registers the ID, saves the current timestamp, and returns a lock approval.

3. Automatic TTL & Storage Alarms

To prevent the Durable Object's persistent storage from growing indefinitely and consuming unnecessary memory:

- The DO registers an **atomic alarm** scheduled for 24 hours in the future.
- When the alarm fires, the DO runs an automatic garbage collection script that purges old IDs from its local storage.
- This ensures that while you are 100% protected against duplicates during network dropouts, your storage footprint remains lightweight.

4. Cold-Start Resilience

When a DO instance has been idle (no requests for the TTL period), Cloudflare may evict it from memory. Upon the next request:

1. The DO is **reconstructed** from its persisted SQLite state on disk.
2. All previously recorded trace IDs are loaded back into memory.
3. The dedup check continues seamlessly — no data loss, no duplicate risk.

This means idempotency protection **survives worker restarts, cold starts, and infrastructure failovers** without any manual intervention.



PERFORMANCE IMPACT

Metric	Value	Notes
DO ping overhead	< 2ms	Per-request latency added
Lock acquisition	< 1ms	Single-threaded, no contention
Dedup check (cache hit)	< 0.5ms	In-memory SQLite query
Storage alarm cleanup	< 5ms	Runs in background, non-blocking

 Warning

Never disable idempotency check bindings in your `wrangler.jsonc` file in production. The performance cost of pinging the DO is less than **2 milliseconds**, while the cost of a duplicate order could be catastrophic.

 Next Steps

- **Signals & Trade Specifications** — Learn how to configure your Pine Script webhook payloads to transmit unique idempotency keys.
- **Platform Security Guides** — Deepen your understanding of Zero Trust headers and edge firewall configurations.



SIGNALS & TRADE SPEC

enduser/concepts/signals-and-trades.md



SIGNALS & TRADE SPEC

This document details the exact specifications, JSON validation schemas, and internal translation logic that occurs when an external trade signal (such as a TradingView alert or Email parser) is ingested by the Hoox Gateway and mapped to an active exchange order.

1. WEBHOOK SIGNAL INGESTION SCHEMA

Every signal received by the public gateway at `/webhook` must contain a valid, authenticated JSON payload. Below is the strict TypeScript interface and parameter schema validated by the gateway's validation middleware:

```
export interface WebhookSignal {  
  apiKey: string;           // Authentication token matching CONFIG_KV webhooks:api_key  
  exchange: "binance" | "bybit" | "mexc"; // Target exchange router  
  action: "LONG" | "SHORT" | "CLOSE";      // Position intent  
  symbol: string;           // Asset symbol (e.g. "BTCUSDT", "ETHUSDT")  
  quantity: number;         // Order size (amount of asset or contracts)  
  leverage?: number;        // Optional leverage multiplier (default: 1, max: 125)  
  idempotencyKey?: string;  // Optional unique signature for dedup (auto-generated if omitted)  
  price?: number;           // Optional limit price (default: market price)  
  takeProfit?: number;      // Optional take-profit price target  
  stopLoss?: number;        // Optional stop-loss price target  
}
```

Parameter Rules & Type Constraints

Parameter	Type	Required	Constraints
<code>apiKey</code>	<code>string</code>	Yes	Must match the encrypted <code>webhooks:api_key</code> hash in your <code>CONFIG_KV</code> namespace
<code>exchange</code>	<code>string</code>	Yes	One of: <code>binance</code> , <code>bybit</code> , <code>mexc</code> — must match a configured exchange router
<code>action</code>	<code>string</code>	Yes	One of: <code>LONG</code> (buy/open), <code>SHORT</code> (sell/open), <code>CLOSE</code> (flatten position)
<code>symbol</code>	<code>string</code>	Yes	Parsed and standardized — <code>BTC-USDT</code> , <code>BTC_USDT</code> , and <code>btc/usdt</code> all become <code>BTCUSDT</code>
<code>quantity</code>	<code>number</code>	Yes	Must be greater than zero; checked against exchange minimum order size
<code>leverage</code>	<code>number</code>	No	Range: <code>1</code> to <code>125</code> depending on exchange margin profiles; defaults to <code>1</code> (spot)
<code>idempotencyKey</code>	<code>string</code>	No	If omitted, auto-generated from <code>SHA256(symbol + exchange + action + timestamp)</code>
<code>price</code>	<code>number</code>	No	Limit price; defaults to current market price if omitted
<code>takeProfit</code>	<code>number</code>	No	Target exit price for automatic partial close
<code>stopLoss</code>	<code>number</code>	No	Stop-loss trigger price; overrides agent-worker trailing stop if specified

2. DYNAMIC PAYLOAD TRANSLATION & SIDE MAPPING

Exchanges do not understand actions like `LONG`, `SHORT`, or `CLOSE`. They process order instructions in terms of `Side` (`BUY` or `SELL`) and `PositionSide` (`LONG` or `SHORT` for multi-margin hedge modes).

The `trade-worker` parses the incoming signal and translates the business logic into exchange-specific API calls:

A. Translation Table (One-Way Margin / Spot)

Action	Position State	Translated Exchange Side	Operation Type
LONG	Closed / No Position	BUY	Opens a long position (spot or margin)
SHORT	Closed / No Position	SELL	Opens a short position (margin/futures)
CLOSE	Long Position Active	SELL	Closes and flattens an existing long position
CLOSE	Short Position Active	BUY	Closes and flattens an existing short position

B. Hedge-Mode PositionSide Mapping

When running in hedge mode (supported on Bybit and Binance futures), both LONG and SHORT positions can coexist:

Hedge Mode:

LONG PositionSide = LONG → BUY opens LONG, SELL closes LONG
SHORT PositionSide = SHORT → SELL opens SHORT, BUY closes SHORT

One-Way Mode:

BUY always opens/increases, SELL always closes/reduces

C. Dynamic Position Resolution

When the action is **CLOSE**, the **trade-worker** automatically performs a sub-millisecond edge check:

1. Queries your local D1 transaction ledger to resolve the active position direction for the symbol.
2. If no position is tracked locally, it performs a real-time portfolio balance check against the exchange's private position endpoint.
3. Automatically sets the order quantity to match your current open exposure, ensuring a perfect, slippage-free execution that flattens the position without leaving residual micro-contracts.

3. LEVERAGE SCALING & ORDER MATH

If the signal includes a **leverage** parameter greater than **1** (e.g., **"leverage": 10**), the **trade-worker** automatically executes a secure margin transition sequence:

1. **Set Margin Mode:** Configures the symbol's margin structure (Isolated vs. Cross) via the exchange API, matching your manifest defaults.
2. **Set Leverage Coefficient:** Submits a leverage update payload to the exchange prior to routing the order.

3. **Calculate Collateral:** If the order size is defined in USDT terms, the worker scales the execution quantity mathematically:
$$\text{Contract Quantity} = \frac{\text{Order Size (USDT)}}{\text{Leverage} \times \text{Current Market Price}}$$
4. **Precision Rounding:** Automatically rounds the calculated quantity down to match the exchange's strict asset decimal precision requirements, preventing API rejects.

Precision Table by Exchange

Exchange	Price Precision	Quantity Precision	Min Order Size
Binance	Varies by symbol	Varies by symbol	~\$10 equivalent
Bybit	0.1 – 0.001	3–6 decimal places	Varies by tier
MEXC	0.1 – 0.00000001	2–8 decimal places	~\$5 equivalent

4. D1 DATABASE TRANSACTION LEDGER

Every filled order is persistently written to your globally distributed edge SQLite table. The schema ensures a clean audit log:

```
CREATE TABLE trades (
  id TEXT PRIMARY KEY,           -- UUID v4 generated by trade-worker
  request_id TEXT NOT NULL,      -- Distributed trace ID from gateway
  exchange TEXT NOT NULL,       -- 'bybit', 'binance', or 'mexc'
  symbol TEXT NOT NULL,         -- e.g. 'BTCUSDT'
  action TEXT NOT NULL,         -- 'LONG', 'SHORT', or 'CLOSE'
  side TEXT NOT NULL,           -- 'BUY' or 'SELL' (exchange-side)
  quantity REAL NOT NULL,       -- Filled asset quantity
  price REAL NOT NULL,          -- Execution entry price (USDT)
  leverage INTEGER DEFAULT 1,   -- Applied leverage multiplier
  fee REAL NOT NULL,            -- Exchange transaction fees (USDT)
  order_id TEXT NOT NULL,       -- Exchange-provided order hash
  status TEXT NOT NULL,         -- 'Filled', 'Rejected', 'Failed', 'Partial'
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE positions (
  id TEXT PRIMARY KEY,
  symbol TEXT NOT NULL UNIQUE,
  side TEXT NOT NULL,           -- 'LONG' or 'SHORT'
  quantity REAL NOT NULL,
  entry_price REAL NOT NULL,
  mark_price REAL,              -- Last known market price
  leverage INTEGER DEFAULT 1,
  unrealized_pnl REAL DEFAULT 0,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

You can query the ledger at any time from your local terminal:

```
# Query the 10 most recent trades in your D1 database
hoox db query "SELECT created_at, exchange, symbol, action, price, quantity FROM trades ORDER BY created_at DESC LIMIT 10"

# Check open positions
hoox db query "SELECT symbol, side, quantity, entry_price FROM positions WHERE quantity > 0"
```

5. QUEUE FAILOVER & GUARANTEED DELIVERY

When the exchange API returns an error (timeout, rate limit, HTTP 5xx), the gateway automatically enqueues the trade payload into **Cloudflare Queues** for guaranteed delivery:

Retry Attempt	Delay	Trigger Condition
1	Immediate	First failure
2	30 seconds	Network timeout
3	1 minute	Rate limit (429)
4	5 minutes	Exchange error (502/503)
5	15 minutes	Extended maintenance
6	30 minutes	Final retry before logging as failed

After max retries, the trade is logged to D1 with `status = 'Failed'` — not lost, but visibly flagged for manual review.

 **Tip**

By utilizing time-series logging via Cloudflare Analytics Engine, Hoox tracks the exact execution duration of each pipeline step. You can audit the latency breakdown (e.g., Gateway auth took `1.2ms` , Trade-worker signing took `0.8ms` , Bybit API roundtrip took `18.5ms`) directly in your Next.js dashboard!

Next Steps

- **Autonomous AI Risk Management** — Learn how agent-worker tracks D1 entries to manage trailing stops and drawdowns.
- **CLI Reference Manual** — Review commands to manage D1 tables, perform dry runs, and tail logs.
- **Error Models & Status Codes** — Understand all gateway error responses.

AI RISK MANAGER & AI GATEWAY

enduser/concepts/ai-risk-manager.md



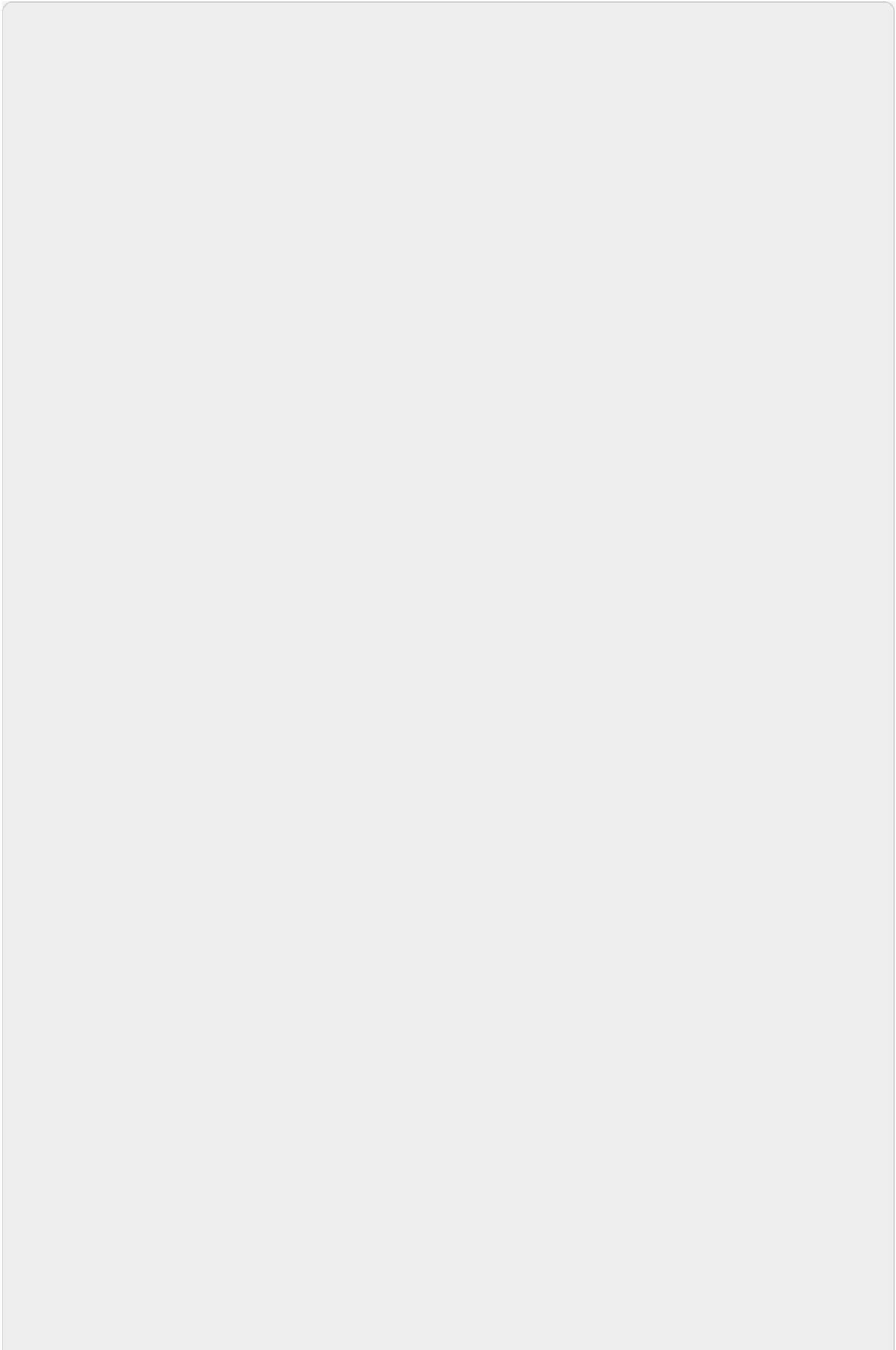
AI RISK MANAGER & AI GATEWAY

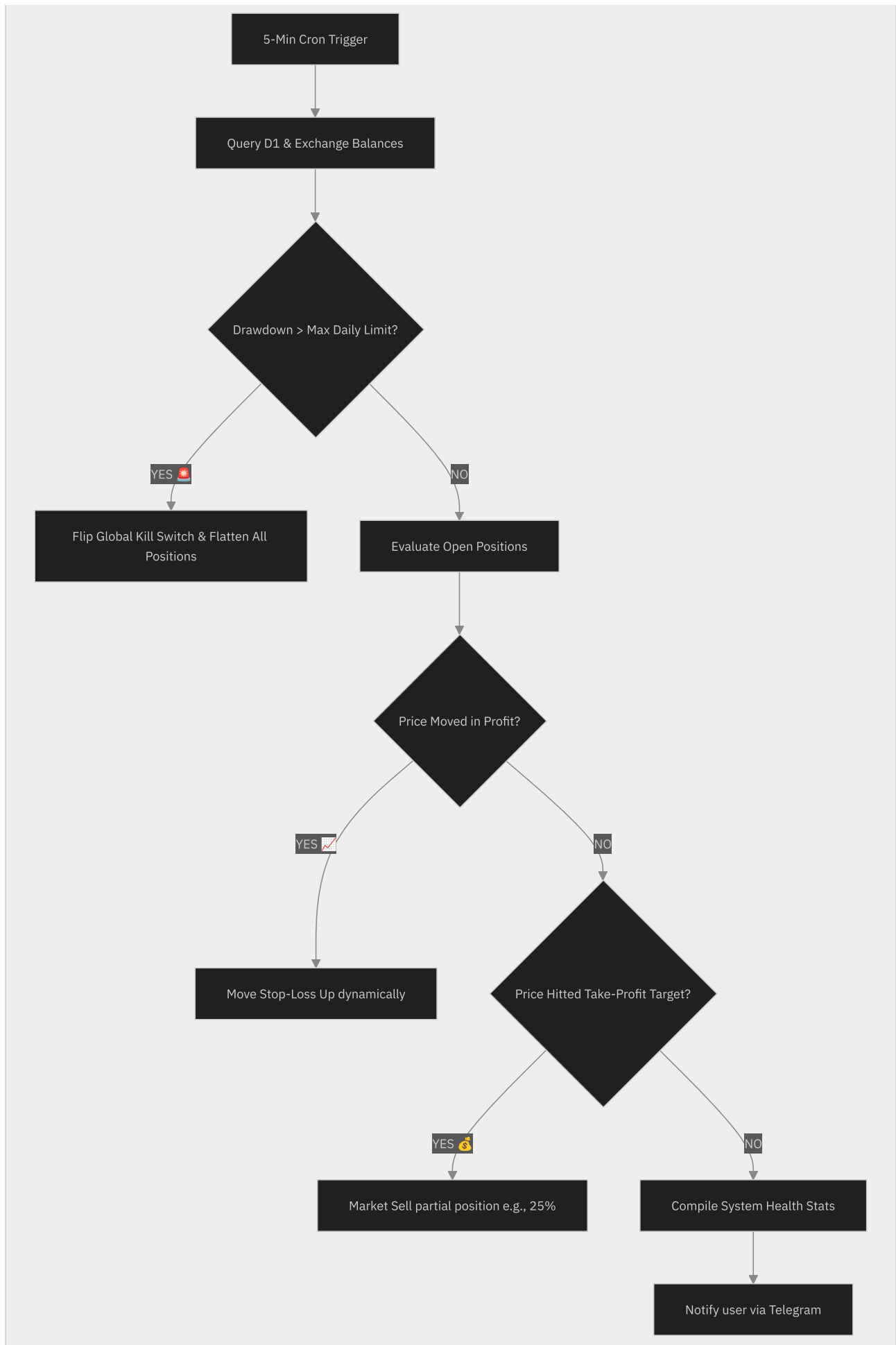
The **agent-worker** is the autonomous central nervous system of the Hoox trading platform. Operating on a continuous **5-minute Cloudflare Cron trigger**, it acts as an intelligent sentinel: it monitors open exchange positions, mathematically scales trailing stop-losses, and deploys a multi-provider fallback AI engine to analyze market conditions, audit risk, and send real-time summaries to your phone.



1. AUTOMATED RISK PROTECTION ENGINE

Every 5 minutes, **agent-worker** executes a multi-point risk diagnostic loop across all active exchanges:





A. Trailing Stop-Loss Mathematics

Rather than using static stops that leave profits exposed, the risk engine calculates a **dynamic trailing stop-loss** at the V8 compiler level:

- **The Formula:** $\text{Trailing Stop Price (Long)} = \text{Peak Price} \times (1 - \text{Trailing Deviation \%})$
- If a LONG trade entry is at \$100 and has a 2% trailing deviation, the initial stop-loss is placed at \$98.
- If the market price rallies to a peak of \$120, the stop-loss is automatically adjusted up to \$117.60.
- If the price declines from \$120 down to \$117.60, the stop is triggered at the exchange level, locking in a \$17.60 profit. The stop never moves downward.

B. Scaled Take-Profits (Partial Fills)

To manage extreme market swings, the agent-worker supports multi-target scaling:

- **Target 1 (3% Profit):** Automatically flattens 25% of open position size.
- **Target 2 (5% Profit):** Flattens another 25% and moves the stop-loss of the remaining 50% to break-even, eliminating downside risk.

C. Max Daily Drawdown & Global Kill Switch

If high volatility causes unexpected stop-outs, the agent-worker aggregates your real-time realized P&L:

1. Calculates cumulative daily loss: $\text{Daily Drawdown \%} = \frac{\text{Starting Daily Balance} - \text{Current Balance}}{\text{Starting Daily Balance}} \times 100$
2. If the drawdown exceeds your KV threshold (e.g. `trade:max_daily_drawdown_percent` is `5`), the agent-worker immediately triggers the **Global Kill Switch** by writing `trade:kill_switch = true` to `CONFIG_KV`.
3. Calls the `trade-worker` service binding to submit immediate market close orders, **flattening all active positions across all exchanges** near-instantaneously.

```
# Check current Kill Switch status via CLI
hoox monitor kill-switch show

# Manually override to halt all trade processes during extreme macro events
hoox monitor kill-switch on

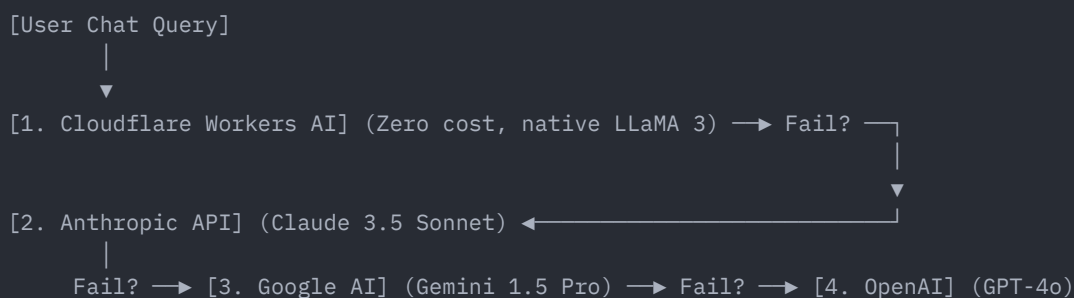
# Resume operations once conditions stabilize
hoox monitor kill-switch off
```



2. MULTI-PROVIDER AI GATEWAY & REASONING

Behind the scenes, `agent-worker` governs a **Multi-Provider AI Gateway** (supporting 5 major providers) with built-in resilience. If you query the bot for trade advice, market summaries, or risk audits, the gateway processes the request through an automatic fallback chain:

A. The Resilient Provider Fallback Chain



B. Custom Telemetry & Chat Endpoints

The gateway exposes professional endpoints for secure inter-worker and external integrations:

- `POST /agent/chat` — Accepts user prompts and handles SSE (Server-Sent Events) streaming responses.
- `POST /agent/vision` — Analyzes charts, balance screenshots, or trading signals from Base64 images.
- `POST /agent/reasoning` — Extended thinking queries with reasoning models (o1, DeepSeek). Respects `thinking_effort` levels.
- `GET /agent/usage` — Detailed token usage and cost metrics across all 5 providers, including per-provider input/output tokens and cumulative USD cost.
- `GET /agent/health` — Health check for all AI providers — returns latency and status for each provider in the fallback chain.

 Tip

You can adjust all AI gateway defaults, Cron check intervals, and trailing deviations in real-time by writing to your KV configurations. No code deployments are ever required to tune your risk profile.

 Next Steps

- **Zero Trust & Secret Security** — Lock down your AI endpoints and exchange API keys.
- **TUI Operations Cockpit** — View live position drawdowns and trailing stops visually in your terminal.

LOCAL DEVELOPMENT

enduser/guides/local-development.md



LOCAL DEVELOPMENT

Hoox provides a comprehensive local development workspace designed to match your production Cloudflare edge environment. You can run all microservices with hot-reload enabled, monitor them visually via the Terminal UI (TUI), and execute the full test suite using native Bun tools or isolated Docker containers.



STARTING THE LOCAL WORKSPACE

To spin up all enabled workers simultaneously:

```
hoox dev start
```

On execution, the CLI automatically detects your environment and prompts you to select a runtime:

Option A: Native Runtime (Recommended for speed)

- Runs each worker in a separate background thread using `wrangler dev` (the official Cloudflare local server).
- **Speed:** Instant startup and sub-millisecond hot-reloading.
- **Requirements:** Local node/bun installation.

Option B: Docker Runtime (Recommended for isolation)

- Launches a multi-container stack using **Docker Compose**.
- **Isolation:** All environment variables, SQLite databases, and queue handlers run in isolated Linux containers, ensuring zero conflicts with local packages.
- **Requirements:** Docker Desktop installed.



Tip

The CLI saves your runtime preference inside `wrangler.jsonc.dev.runtime`. Subsequent launches skip the prompt. You can override your preference at any time using flags:

```
# Force native execution
hoox dev start --runtime native

# Force Docker Compose execution
hoox dev start --runtime docker
```

DOCKER COMPOSE PROFILES

If you choose the Docker runtime, Hoox manages orchestration using three specialized Docker Compose profiles defined in `docker-compose.yml` :

```
# Profile 1: Workers Only (No frontend dashboard)
docker compose --profile workers up

# Profile 2: Dashboard Only (Next.js server only)
docker compose --profile dashboard up

# Profile 3: Full Stack (All workers + Next.js dashboard)
docker compose --profile full up
```

LOCAL PORT MAPPING & ENDPOINT ACCESS

During local development, all enabled workers are assigned dedicated local ports, simulating service boundaries locally:

Worker	Local Port	Endpoint URL	Purpose
hoox	8787	http://localhost:8787	Public Gateway & Webhook Receiver
trade-worker	8789	http://localhost:8789	Trade Execution Engine
telegram-worker	8791	http://localhost:8791	Telegram Bot Alerts & Commands
d1-worker	8792	http://localhost:8792	SQLite Database Operations
web3-wallet	8793	http://localhost:8793	On-Chain DeFi Execution
dashboard	8794	http://localhost:8794	Next.js Dashboard Cockpit
agent-worker	8795	http://localhost:8795	AI Risk Manager & Cron Engine
email-worker	8796	http://localhost:8796	Email Signal Parsing
report-worker	8797	http://localhost:8797	PDF Portfolio Report Generator

OPERATING INDIVIDUAL SERVICES

If you only want to work on a single microservice rather than running the full stack, you can spin up individual modules:

```
# Dev run a single worker (gateway)
hoox dev worker hoox

# Dev run trade-worker with hot-reloading
hoox dev worker trade-worker

# Dev run dashboard separately (Next.js dev server with Turbopack)
hoox dev dashboard
```



RUNNING THE VERIFICATION CI PIPELINE

Hoox features a rigorous local test pipeline to ensure that all TypeScript types, formatting, and unit tests pass perfectly before pushing to git:

```
# Run the complete CI verification pipeline locally
hoox test
```

The pipeline executes four verification steps in a strict dependency sequence:

1. **Lint Check** (`bun run lint`): Validates ESLint styling rules across the monorepo.
2. **Type Check** (`bun run typecheck`): Compiles code via `tsc --noEmit` to verify type safety.
3. **Unit Tests** (`bun test`): Fires all unit and integration test assertions using Bun's native test runner.
4. **Build Check** (`bun run build`): Compiles all workspaces (`cli` , `tui` , `shared` , `dashboard`) to verify production packaging.

```
🔍 Running local CI Pipeline...
[STARTED] lint check... [PASSED]
[STARTED] TypeScript typecheck... [PASSED]
[STARTED] bun test runner... [PASSED] (1,574 assertions)
[STARTED] workspace builds... [PASSED]
✓ Local CI Pipeline Succeeded!
```



Note

Local unit tests utilize Bun's native test runner for instantaneous execution. You can target specific workspace folders or run files individually: `bun test workers/trade-worker/src/index.test.ts` .



Next Steps

- **Testing Standards** — Run unit and integration tests using Bun's native test runner.
- **Terminal UI Operations** — Launch the full-screen terminal cockpit (`./hoox-tui`) to monitor these local runs.

- **Database Migrations** — Set up, query, and migrate your local SQLite D1 database.

TERMINAL UI (TUI)

enduser/guides/tui.md



TERMINAL UI (TUI)

The **Hoox Terminal User Interface (TUI)** is a full-screen, keyboard-driven operations cockpit built natively with **OpenTUI**, React 19, and Zustand state stores. Designed for command-line efficiency, the TUI provides real-time visibility into your entire distributed trading infrastructure—allowing you to inspect compute instances, tail logs, query D1 databases, manage configurations, and deploy edge workers in seconds.



LAUNCHING THE TUI

Launch the cockpit directly from your terminal using any of the following methods:

```
# 1. Recommended: Shell script launcher (from project root)
./hoox-tui

# 2. Recommended: Via the hoox CLI tool
hoox tui

# 3. Development: Run direct hot-reload development server
cd packages/tui && bun run dev

# 4. Flags: Override frame rate and disable mouse interaction
hoox tui --fps 60 --no-mouse
```



GLOBAL KEYBOARD NAVIGATION CHEATSHEET

The TUI features a **keyboard-first layout**. All shortcuts are registered globally and can be triggered from any active view:

Keyboard Shortcut	Operation	View Name
Ctrl+1	Jump to View	Dashboard — System Health Overview
Ctrl+2	Jump to View	Workers Overview — 2-Column Cards Grid
Ctrl+3	Jump to View	Worker Detail — Deep-Dive Metrics
Ctrl+4	Jump to View	Trade Monitor — Real-Time Transaction Feed
Ctrl+5	Jump to View	Logs Viewer — Streaming Log Console
Ctrl+6	Jump to View	Service Manager — Deploy & Rebuild Controls
Ctrl+7	Jump to View	Config Editor — Syntax-Highlighted TOML/JSON Editor
Ctrl+8	Jump to View	Setup Wizard — Onboarding Flow
Ctrl+9	Jump to View	Settings — UI Preferences & theme toggle
Ctrl+P	Action	Command Palette — Fuzzy-search search overlay
Ctrl+B	Action	Toggle Sidebar — Show/Hide menu
Ctrl+R	Action	Force Refresh — Recalculate metrics
Ctrl+Q	Action	Quit — Displays exit confirmation dialog
Esc	Action	Close overlay / Dismiss modal / Go back
Tab / Shift+Tab	Navigation	Cycle active focus between elements
Enter	Navigation	Select / Execute / Toggle button

 COCKPIT TOUR: THE 9 CORE VIEWS

1. The Operations Dashboard (Ctrl+1)

Your high-signal, single pane of glass.

- **Service Status Grid:** Operational, Degraded, or Offline indicators for all 9 edge workers.
- **Telemetry Statistics:** Session P&L, cumulative win rates, Sharpe ratio, and total API counts.
- **Alert Box:** Color-coded, scrolling logs of warnings, drawdowns, and security events.

2. Workers Overview (Ctrl+2)

A cards-based visual grid representing every running isolate.

- Displays worker status, uptime metrics, CPU cycles (ms), and active RAM footprint (MB).
- Shows active **Durable Object instances** (locks) and the number of processed signals.
- Selecting a worker and hitting **Enter** opens the **Worker Detail** deep-dive.

3. Worker Detail (**Ctrl+3**)

A 4-quadrant analytical layout focused on a single worker:

- **Top Left: Metrics:** Graphs representing memory allocation and request velocities over time.
- **Top Right: Live Logs:** Scrollable, real-time log terminal (press **p** to pause logging).
- **Bottom Left: Durable Objects:** SQLite state indexes and active DO alarm TTL counts.
- **Bottom Right: Config Preview:** Snapshot of active `wrangler.jsonc` binds.

4. Trade Monitor (**Ctrl+4**)

The financial ledger center.

- **Live Feed:** Scrolling record of recent fills (Symbol, Side, Execution Price, Contracts, P&L).
- **Open Positions:** Grouped active exposure showing current size, entry price, liquidation boundaries, and real-time unrealized P&L.
- **Win Metrics:** Graphical gauges for win/loss ratio, average trade duration, and daily performance metrics.

5. Logs Viewer (**Ctrl+5**)

A centralized console debugger:

- **Filter Panel:** Multi-select checkboxes for severity filters (**INFO** , **WARN** , **ERROR**), target worker select, and a fuzzy text search box.
- **Log Stream:** Scrolling terminal window showing formatted, color-coded lines.

6. Service Manager (**Ctrl+6**)

Your deployment pipeline:

- **Worker Controls:** Trigger direct deployments (`hoox deploy`) or soft restarts (`wrangler dev` resets) on individual workers.
- **Interactive Edge Map:** Displays locations of Cloudflare Points of Presence (PoPs) globally with status checks.
- **Bulk Executions:** Re-deploy the entire ecosystem in one click, secured by confirmation modals.

7. Configuration Editor (**Ctrl+7**)

An IDE-class terminal file editor:

- **Syntax Highlighting:** Supports full color-coded rendering for TOML and JSON structures.
- **Live Linter:** Evaluates brackets, balancing quotes, and JSON structures on the fly, showing error locations.
- **Prettifier:** Automatically formats code blocks (spacing, indentation) on save (**Ctrl+S**).

8. Guided Setup Wizard (**Ctrl+8**)

Onboards new machines in 6 steps:

1. **API Keys:** Authenticate Cloudflare credentials.
2. **Exchanges:** Inject Bybit, Binance, or MEXC trade API keys.
3. **AI Credentials:** Set up multi-provider keys (OpenAI, Gemini, Anthropic).
4. **Strategies:** Configure default margins and symbols.
5. **Telegram Bot:** Link tokens and Chat IDs.
6. **Kickoff:** Initiate deployments.

9. Cockpit Settings (**Ctrl+9**)

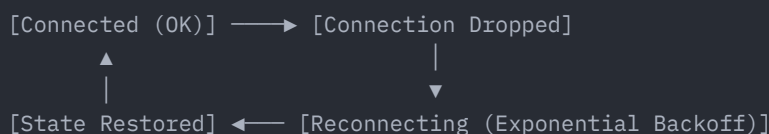
Configure cockpit parameters:

- **Theme:** Swap color themes in real-time.
- **Telemetry:** Set metrics refresh rates (default: every 2 seconds).
- **Reset:** Clean local TUI caching (`~/.hoox/session.json`).



OFFLINE CONNECTION RESILIENCE STORE

The TUI utilizes an advanced Zustand state machine to govern connectivity to your local API dev server or remote workers:



- **States:** **Connected** (Green dot), **Reconnecting** (Yellow/Orange pulsing dot), **Offline** (Red/Empty dot).

- **Resilience:** In the event of network dropouts, the TUI activates an exponential backoff engine (starting at `500ms`, doubling up to `16s`), trying to reconnect in the background without freezing or crashing the terminal display.



TROUBLESHOOTING & TERMINAL COMPATIBILITY

alternate screen buffer cleanup

If you force-close the terminal and find that your prompt remains garbled, execute a terminal reset:

```
reset  
# or  
tput reset
```

“Ctrl+Q” is intercepted by Flow Control

If `Ctrl+Q` (Quit) fails to trigger, your terminal emulator is likely intercepting flow control commands (XON/XOFF). Disable software flow control by running this command before launching the TUI:

```
stty -ixon
```

Garbled layout or text overlaps

The TUI requires a minimum screen resolution of **80 columns × 24 rows**. If your terminal is too small, resize the window. Additionally, ensure your system `TERM` variable is configured to support 256 colors:

```
export TERM=xterm-256color
```



Next Steps

- **Local Development Guides** — Spin up the API dev server that feeds the TUI.
- **CLI Reference Manual** — Read the commands running under the TUI hood.



DATABASE OPERATIONS

enduser/guides/database-ops.md



DATABASE OPERATIONS

Hoox utilizes **Cloudflare® D1**—a fully serverless, highly optimized SQLite database engine distributed globally across Cloudflare’s edge network. This document serves as your operational runbook for executing database schemas, running schema migrations, querying transaction ledgers, and performing secure database backup and recovery.



THE CORE DATABASE TABLES

The database schema defines five fundamental tables designed for trading operations, plus supporting indices for fast query performance:

Table	Purpose	Row Estimate (Active User)
trades	The primary ledger. Execution prices, contract quantities, timestamps, fees, and exchange order IDs.	~10,000/mo
positions	Tracks open margin/futures exposure (average entry price, size, leverage, direction).	~50–200
balances	Periodic snapshots of account equity, margin balance, and available free collateral.	~30/day
trade_signals	Historical record of every raw incoming webhook alert before execution processing.	~10,000/mo
system_logs	Crucial debug and error messages offloaded from compute nodes.	~5,000/mo

Schema Details

```
-- Core trades ledger
CREATE TABLE trades (
  id TEXT PRIMARY KEY,
  request_id TEXT NOT NULL,
  exchange TEXT NOT NULL,          -- 'bybit', 'binance', 'mexc'
  symbol TEXT NOT NULL,
  action TEXT NOT NULL,            -- 'LONG', 'SHORT', 'CLOSE'
  side TEXT NOT NULL,              -- 'BUY' or 'SELL'
  quantity REAL NOT NULL,
  price REAL NOT NULL,
  leverage INTEGER DEFAULT 1,
  fee REAL NOT NULL DEFAULT 0,
  order_id TEXT NOT NULL,
  status TEXT NOT NULL,            -- 'Filled', 'Rejected', 'Failed', 'Partial'
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Active positions tracking
CREATE TABLE positions (
  id TEXT PRIMARY KEY,
  symbol TEXT NOT NULL UNIQUE,
  side TEXT NOT NULL,
  quantity REAL NOT NULL,
  entry_price REAL NOT NULL,
  mark_price REAL DEFAULT 0,
  leverage INTEGER DEFAULT 1,
  unrealized_pnl REAL DEFAULT 0,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Account balance snapshots
CREATE TABLE balances (
  id TEXT PRIMARY KEY,
  total_balance REAL NOT NULL,
  available_balance REAL NOT NULL,
  unrealized_pnl REAL DEFAULT 0,
  snapshot_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Incoming signal audit trail
CREATE TABLE trade_signals (
  id TEXT PRIMARY KEY,
  raw_payload TEXT NOT NULL,
  parsed_symbol TEXT,
  parsed_action TEXT,
  received_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- System diagnostic logs
CREATE TABLE system_logs (
  id TEXT PRIMARY KEY,
  level TEXT NOT NULL,              -- 'info', 'warn', 'error'
  source TEXT NOT NULL,             -- Worker name
  message TEXT NOT NULL,
  metadata TEXT,                   -- JSON blob of additional context
);
```



```
created_at TIMESTAMPT DEFAULT CURRENT_TIMESTAMP
);
```

APPLYING SCHEMAS & TABLES

When launching a new workspace, you must initialize the required tables. The Hoox CLI handles this via declarative migration scripts:

```
# 1. Apply schema and seed initial tables to local dev SQLite
hoox db apply

# 2. Deploy schema and seed tables directly to live production Cloudflare D1
hoox db apply --remote
```

Migration Files Location

Migrations are stored in `workers/d1-worker/src/drizzle/` and are automatically versioned:

```
workers/d1-worker/src/drizzle/
├─ 0001_initial_schema.sql
├─ 0002_add_leverage_column.sql
├─ 0003_balances_snapshot_table.sql
├─ meta/
│   └─ snapshot.json
```

Warning

Running `hoox db apply` compiles migrations and seeds the database locally. For production deployment, you **must** append the `--remote` flag to execute operations directly on your active Cloudflare D1 instance.

MANAGING DATABASE MIGRATIONS

When new features are added that require changes to the database structure, you must run migrations:

```
# List all applied and pending database migrations in production
hoox db migrate status --remote

# Apply all pending schema migrations sequentially to production D1
hoox db migrate --remote

# Rollback the most recent migration (use with caution)
hoox db migrate rollback --remote
```

The migration engine tracks history inside a special `d1_migrations` table, ensuring that migrations are never executed twice or applied in the wrong sequence.

Migration Best Practices

1. **Always write reversible migrations** — include both `UP` and `DOWN` scripts
2. **Never modify existing migrations** — create new ones for schema changes
3. **Test locally first** — run `hoox db apply` without `--remote` to verify
4. **Backup before migrating** — export your production database first



QUERYING DATA FROM THE CLI

The Hoox CLI features a built-in SQL interface allowing you to run arbitrary queries directly against your local or remote database:

```
# A. List all active tables in your production database
hoox db list --remote

# B. Count the total number of executed trades
hoox db query "SELECT COUNT(*) FROM trades" --remote

# C. Inspect the 5 most recent trade fills with formatting
hoox db query "SELECT created_at, symbol, action, price, quantity FROM trades ORDER BY created_at DESC LIMIT 5" --remote

# D. Check currently open positions on Bybit
hoox db query "SELECT symbol, side, size, entry_price FROM positions WHERE size > 0" --remote

# E. Calculate daily P&L
hoox db query "SELECT DATE(created_at) as day, SUM(CASE WHEN side='BUY' THEN -price*quantity ELSE price*quantity END) as pnl FROM trades GROUP BY day" --remote

# F. Export trades as CSV
hoox db query "SELECT * FROM trades ORDER BY created_at DESC" --remote --format csv > trades_export.csv
```

Useful Query Templates

```
-- Win rate calculation
SELECT
  COUNT(CASE WHEN realized_pnl > 0 THEN 1 END) * 100.0 / COUNT(*) AS win_rate_pct
FROM trades WHERE status = 'Filled';

-- Average trade size by exchange
SELECT exchange, AVG(quantity * price) as avg_trade_size
FROM trades WHERE status = 'Filled'
GROUP BY exchange;

-- Hourly trading distribution (for optimal timing)
SELECT strftime('%H', created_at) as hour, COUNT(*) as trade_count
FROM trades WHERE status = 'Filled'
GROUP BY hour ORDER BY hour;
```



BACKUP & EXPORT WORKFLOWS

To secure your historical P&L records, transaction ledger, and bot performance telemetry, execute regular exports:

```
# Export the entire D1 database as a clean SQL script
hoox db export --remote

# Export to a specific file path
hoox db export --remote --output backups/my-backup.sql
```

Export Output & Structure

The export command creates a timestamped SQL dump inside your workspace directory:

```
backups/db-backup-2026-05-19-174000.sql
```

This file contains standard DDL and DML commands:

```
PRAGMA foreign_keys=OFF;
BEGIN TRANSACTION;
CREATE TABLE IF NOT EXISTS trades (...);
INSERT INTO trades VALUES(...);
COMMIT;
```

Restoring from Backup

```
# 1. Create a fresh database first
hoox infra d1 create hoox-db-backup

# 2. Apply the backup SQL
bunx wrangler d1 execute hoox-db-backup --file=backups/db-backup-2026-05-19.sql
```



DATABASE RESET (DESTRUCTIVE OPERATIONS)

If you are running simulated paper trading and wish to wipe all ledger history to start fresh, you can reset the tables:

```
# Wipe all tables in the local development database
hoox db reset --confirm

# Wipe all tables in the live production D1 database (USE WITH EXTREME CAUTION)
hoox db reset --remote --confirm
```



Danger

Wiping your production D1 database is an irreversible operation. It will permanently delete all trade logs, position records, and asset histories. **Always** execute `hoox db export --remote` before running a reset!

Soft Reset Alternative

If you want to clear data but preserve schema structure:

```
# Truncate all data tables while keeping schema intact
hoox db query "DELETE FROM trades; DELETE FROM positions; DELETE FROM balances; DELETE FROM sys"
```

D1 FREE TIER LIMITS & MONITORING

Resource	Free Tier Limit	Monitoring Command
Rows Read/Day	5,000,000	<code>hoox db query "SELECT COUNT(*) FROM trades WHERE created_at >= date('now', '-1 day')"</code>
Rows Written/Day	100,000	<code>hoox db query "SELECT COUNT(*) FROM trades WHERE created_at >= date('now', '-1 day')"</code>
Storage	5 GB	<code>hoox db query "SELECT page_count * page_size FROM pragma_page_count(), pragma_page_size()"</code>
Max Connections	Concurrent	N/A — serverless, no connection pool

Next Steps

- **Local Development & Testing** — Run your local V8 wrangler isolates with local D1 SQLite bindings.
- **Infrastructure Management** — Manage KV config namespaces, Queue parameters, and R2 storage buckets.
- **Schema Reference** — Detailed DDL documentation and storage engineering details.

SECRETS & NETWORK SECURITY

enduser/guides/secrets-security.md



SECRETS & NETWORK SECURITY

Security is the most critical component of the Hoox trading platform. When deploying automated execution scripts, your capital and API credentials must be protected against malicious exploits, unauthorized webhook payloads, and network interceptions.

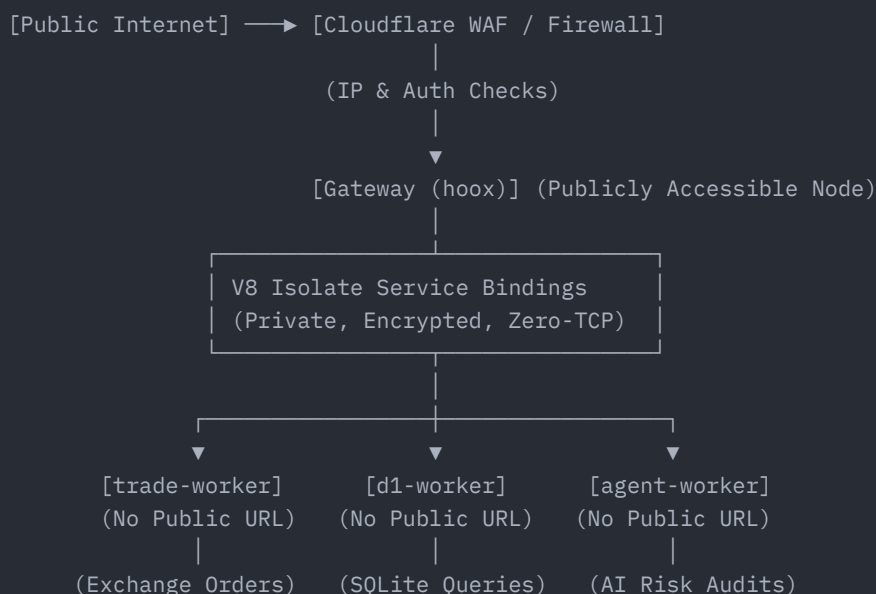
This guide outlines our **Zero Trust security architecture**, encrypted secret management procedures, and edge-level firewall protection runbooks.



1. ZERO TRUST NETWORK ISOLATION

Traditional trading systems expose database and exchange execution APIs to the public internet (secured by simple HTTP headers or ports). This creates an active attack surface.

Hoox implements a strict **Zero Trust microservice isolation topology**:



- **No Public Endpoints:** The `trade-worker`, `d1-worker`, and `agent-worker` literally **do not exist** on the public internet. They have no public IP addresses or URLs.
- **V8 Service Bindings:** Communication between the public gateway (`hoox`) and internal compute nodes is routed entirely inside Cloudflare's secure V8 engine isolates. Your trade routing data, database queries, and private logs never travel over the public internet, eliminating TLS decryption and packet-sniffing risks.



2. ENCRYPTED SECRET MANAGEMENT VIA CLI

API keys, exchange secrets, and Telegram bot tokens are never committed to git repositories or written in plain-text configuration files. Instead, they are stored directly on Cloudflare's hardware-secured key vaults.

The Hoox CLI features deep encryption integrations to automate secret provisioning:

```
# 1. Inject a secure exchange credential (e.g. Bybit Secret)
hoox secrets set BYBIT_API_SECRET "your_private_signature_here"

# 2. Check the synchronization status of all required edge secrets
hoox secrets check
```

The Secrets Diagnostic Report

Running `hoox secrets check` queries Cloudflare's API to confirm that the key binding exists on the edge, without ever exposing or decrypting the actual values in your terminal:

```
Cloudflare Edge Secrets Audit

BYBIT_API_KEY ..... ✓ PRESENT (Active)
BYBIT_API_SECRET ..... ✓ PRESENT (Active)
TELEGRAM_BOT_TOKEN ..... ✓ PRESENT (Active)
OPENAI_API_KEY ..... ⚠ MISSING (Optional)

Audit Result: SECURE (All required secrets bound)
```



3. WEBHOOK FIREWALL & TRADINGVIEW IP ALLOW-LISTING

To ensure that only TradingView's official servers can fire signals to your `/webhook` gateway:

- Passkey Verification:** The gateway checks the `apiKey` property inside the JSON payload against your secure manifest in `CONFIG_KV`.
- Cloudflare WAF (Web Application Firewall):** Since TradingView publishes their official IP ranges, you can configure Cloudflare's edge firewall to block all webhook traffic that does not originate from these verified IPs.

```
# Auto-configure WAF rules to lock the /webhook route to TradingView IPs
hoox waf configure --TradingView-only
```

4. SECURITY BEST PRACTICES CHECKLIST

- **Least Privilege API Keys:** When creating API keys on Bybit, Binance, or MEXC, **never** enable “Withdrawal” permissions. Only check “Trade” and “Account Read” permissions.
- **Credential Rotation:** Automatically rotate your exchange API keys every 90 days. Deleting old keys and injecting new ones takes less than 60 seconds with `hoox secrets set`.
- **Zero-Commit Rule:** Verify that your `.env.local` and `.dev.vars` files are registered in your workspace’s `.gitignore` file to prevent accidental pushes to public repos.
- **Emergency Response:** If you suspect a strategy error or exchange anomaly, immediately halt all execution via the CLI:

```
hoox monitor kill-switch on
```

Next Steps

- | - **Platform Configuration Guide** — Map environment variables, secrets, and KV runtime settings.
- **Local Development & Testing** — Run local wrangler sandboxes with securely encrypted `.dev.vars`.

INFRASTRUCTURE MANAGEMENT

enduser/guides/manage-infra.md



INFRASTRUCTURE MANAGEMENT

Hoox is fully serverless, using Cloudflare’s distributed services as its compute and storage engine. Managing these resources—such as provisioning D1 databases, registering KV config buckets, setting up S3-compatible R2 storage, and mounting asynchronous Queues—is done directly via the **hoox infra** command group.

This guide is your operations manual for provisioning, inspecting, and managing Hoox edge infrastructure.

1. ONE-CLICK QUICK PROVISIONING

When setting up your trading workspace for the first time, you do not need to manually configure resources in Cloudflare’s dashboard. The Hoox CLI automatically parses your enabled workers’ configurations and spins up the entire infrastructure stack in one command:

```
# Auto-provision all required KV, D1, R2, Vectorize, and Queue resources
hoox infra provision
```

This command performs a dependency audit and calls Cloudflare’s APIs to provision:

1. **D1 SQLite Database** (**hoox-db**).
2. **CONFIG_KV** Namespace bucket.
3. **trade-execution** messaging Queue.
4. **trade-reports** and **system-logs** R2 object storage buckets.
5. **rag-index** Vectorize vector search database.



2. D1 DATABASE ADMINISTRATION

D1 databases store your critical transactional data (ledger, positions, balance history).

```
# A. List all active D1 databases in your Cloudflare account
hoox infra d1 list

# B. Create a new custom database instance
hoox infra d1 create my-custom-db

# C. Delete a database instance (destructive)
hoox infra d1 delete my-custom-db
```

**Note**

Once a D1 database is created, the CLI will output its unique `database_id` (a 36-character UUID). You must ensure this ID is bound in your worker's `wrangler.jsonc` file so the worker V8 isolate can establish a connection at runtime.



3. KV CONFIGURATION NAMESPACE MANAGEMENT

KV stores are used to manage fast-path configurations, global parameters, and the kill switch.

```
# A. List all KV namespaces in your account
hoox infra kv list

# B. Create a new KV namespace (e.g. for staging or production)
hoox infra kv create CONFIG_KV

# C. Apply the default 16-key configuration manifest to your active namespace
hoox config kv apply-manifest
```



4. ASYNCHRONOUS QUEUES SETUP

Queues guarantee order delivery during periods of extreme exchange rate limits or network congestion.

```
# A. List all active Cloudflare Queues
hoox infra queues list

# B. Create the trade execution queue
hoox infra queues create trade-execution
```

Queue Parameters & Throttling

During provisioning, Hoox configures the queue with:

- **Retry Backoff:** 30 seconds initial delay, scaling exponentially up to 15 minutes.

- **Message Retention:** 4 days (ensuring messages are preserved even during prolonged exchange maintenance events).



5. R2 OBJECT STORAGE ADMINISTRATION

R2 is an S3-compatible, zero-egress fee object storage service used to offload heavy JSON payloads and PDF portfolio reports.

```
# A. List all R2 buckets
hoox infra r2 list

# B. Create the trade-reports bucket
hoox infra r2 create trade-reports
```



6. VECTORIZE RAG INDEX MANAGEMENT

Vectorize indexes house high-dimensional vector embeddings that power semantic search and Telegram AI bot memory.

```
# A. List all Vectorize indexes
hoox infra vectorize list

# B. Create a vector index with specific dimensions (e.g. 1536 for OpenAI embeddings)
hoox infra vectorize create rag-index --dimensions 1536 --metric cosine
```

⚠ Warning

Destroying resources via `hoox infra <service> delete` is highly destructive and irreversible. Deleting a D1 database instantly wipes your trading records; deleting a KV bucket drops all configurations and triggers immediate gateway errors. Always backup ledgers before running deletions!



Next Steps

- **Database Migrations & SQL** — Run queries, manage drizzle schemas, and restore backups.
- **Take-to-Production Deployments** — Deploy your compiled workers and bind V8 isolates globally.



DEPLOYING TO PRODUCTION

enduser/guides/deploy-workers.md



DEPLOYING TO PRODUCTION

Deploying your algorithmic trading ecosystem to Cloudflare’s production edge requires careful orchestration. Because workers communicate internally using fast-path **Service Bindings**, they have strict compile-time and deploy-time dependencies.

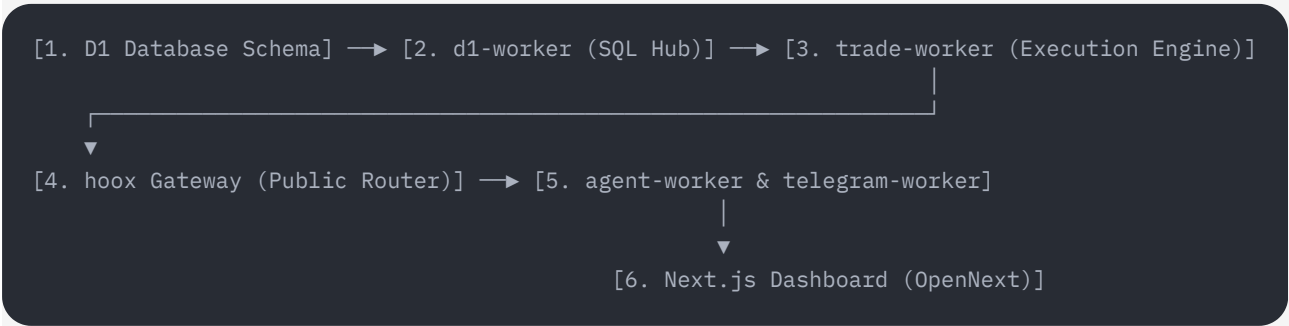
This guide outlines our automated deployment sequence, Next.js OpenNext compilation steps, post-deployment URL bindings, and troubleshooting runbooks.



THE STRICT DEPLOYMENT DEPENDENCY CHAIN

When you deploy, workers must be uploaded in a specific sequence. If you attempt to deploy the public gateway (`hoox`) before the database or execution engines are live, the deployment will fail because the gateway’s compile-time service bindings cannot resolve their targets.

The Hoox CLI automates this dependency hierarchy:



Deployment Order Explained

Step	Worker	Why First?
1	D1 Schema	Database tables must exist before workers try to read/write
2	d1-worker	SQL hub must be available for other workers’ DB operations
3	trade-worker	Core execution engine — other workers depend on it
4	hoox Gateway	Public-facing router — needs all internal workers live
5	agent-worker + telegram-worker	Background services — can deploy independently
6	Dashboard (OpenNext)	Frontend — depends on gateway and all APIs

DEPLOYMENT CLI COMMANDS

To execute a complete production rollout:

```
# 1. Compile and deploy all enabled workers in the correct dependency order
hoox deploy all --auto

# 2. Deploy a single specific worker (e.g. after a custom logic update)
hoox deploy worker trade-worker

# 3. Redeploy only the dashboard
hoox deploy dashboard
```

Deploy All Flags

Flag	Description	Example
<code>--auto</code>	Automatically resolves dependency order	<code>hoox deploy all --auto</code>
<code>--skip-tests</code>	Skip pre-deploy test suite	<code>hoox deploy all --auto --skip-tests</code>
<code>--dry-run</code>	Preview deployment plan without uploading	<code>hoox deploy all --auto --dry-run</code>
<code>--workers-only</code>	Skip dashboard deployment	<code>hoox deploy all --auto --workers-only</code>

CRITICAL POST-DEPLOYMENT STEPS

Once `hoox deploy all` finishes uploading the workers, you must run three final scripts to link webhooks and sync environment databases:

```
# A. Register your private Telegram Bot webhook with Cloudflare
hoox deploy telegram-webhook

# B. Auto-update and bind internal Service URLs across all workers
hoox deploy update-internal-urls

# C. Sync and apply the default CONFIG_KV manifest variables
hoox deploy kv-config
```

Why These Steps Are Required

1. **Telegram Webhook Registration** — Tells Telegram's Bot API where to send user messages. Without this, your bot will be unreachable.
2. **Internal URL Binding** — After deployment, each worker gets a new URL. This command updates all Service Binding references so workers can find each other.
3. **KV Manifest Sync** — Ensures your runtime configuration (kill switch, rate limits, routing rules) matches the freshly deployed workers.



NEXT.JS DASHBOARD DEPLOYMENT (OPENNEXT)

The Hoox Command Center is a modern Next.js 16 web application that runs directly on Cloudflare Workers using the **OpenNext adapter**.

When you run `hoox deploy dashboard` :

1. The CLI compiles the Next.js app using the **Turbopack** build engine.
2. The **OpenNext** compiler bundles the application logic into a single Edge-compliant V8 worker file: `.open-next/worker.js`.
3. All static files (images, JS chunks, CSS) are isolated under `.open-next/assets/`.
4. The CLI uses `wrangler` to deploy the worker and binds the static assets to Cloudflare's **ASSETS binding**, serving pages at sub-millisecond speeds.

```
# Build and deploy the Next.js Dashboard to Cloudflare Workers
hoox deploy dashboard

# Rebuild dashboard only (after UI changes)
hoox deploy dashboard --force-rebuild
```

Dashboard Features

Once deployed, the Command Center provides:

- Real-time portfolio value and P&L tracking
- Win rate statistics and trade history charts
- Live position monitoring with one-click close
- Risk settings adjustment (no redeploy needed)
- AI-powered trade analysis chat



ROUTINE UPDATE & UPGRADE WORKFLOW

To pull the latest platform releases, run local tests, and update your active production edge:

```
# 1. Fetch latest changes and update git submodules recursively
git pull --recurse-submodules

# 2. Install updated dependencies
bun install

# 3. Execute the full local CI verification pipeline
hoox test

# 4. Run pre-deploy health check
hoox repair check

# 5. Roll out updates globally to the Cloudflare Edge
hoox deploy all --auto
```

Rollback Procedure

If the new deployment causes issues:

```
# 1. Revert git to previous release
git checkout v2.0.0 # or your previous tag

# 2. Redeploy the stable version
hoox deploy all --auto
```



POST-DEPLOYMENT TROUBLESHOOTING MATRIX

Error Code / Symptom	Primary Root Cause	Guided Resolution
502 Bad Gateway	Service binding target is missing or has crashed.	Ensure the dependency worker (e.g., <code>trade-worker</code>) has been deployed successfully. Run <code>hoox deploy all</code> to rebuild all bonds.
503 Service Unavailable	The Global Kill Switch is active in KV.	Verify switch state: <code>hoox monitor kill-switch show</code> . If safe, restore trading: <code>hoox monitor kill-switch off</code> .
401 Unauthorized	Webhook request failed passkey check.	Audit KV authorization key: <code>hoox config kv get webhooks:api_key</code> . Verify the payload <code>apiKey</code> matches this value.
409 Conflict	Durable Object intercepted a duplicate trace ID.	Verify TV alert settings. Do not configure rapid-fire duplicate alerts within the same minute unless idempotency keys are unique.
Dashboard shows 500	Next.js build failed or D1 schema mismatch.	Re-run <code>hoox deploy dashboard</code> and check logs: <code>hoox logs tail dashboard</code> .
Telegram bot unresponsive	Webhook URL expired or token rotated.	Re-register: <code>hoox deploy telegram-webhook</code> and verify token: <code>hoox secrets check</code> .



Tip

By utilizing **Smart Placement** in your production builds, Cloudflare will automatically route execution to the edge nodes located geographically closest to your exchanges (Frankfurt, Tokyo), ensuring high-speed fills!



Next Steps

- **Real-Time Observability & Monitoring** — Audit live fills, tail console logs, and run health diagnostics.
- **System Self-Healing & Repair** — Rebuild broken bindings and recover from deployment anomalies.
- **Infrastructure Management** — Provision and inspect KV, Queues, R2, and Vectorize indexes.



SELF-HEALING & REPAIR

enduser/guides/repair.md



SELF-HEALING & REPAIR

Algorithmic trading environments must be resilient and self-healing. If you experience deployment failures, database routing discrepancies, expired authentication tokens, or missing git submodules, the Hoox CLI features a dedicated **hoox repair** command group designed to diagnose issues, check infrastructure status, and recover your workspace automatically.



1. RUNNING THE 5-STEP SYSTEM DIAGNOSTICS

If your trading gateway throws errors or the dashboard reports connection issues, run a comprehensive diagnostic sweep:

```
# Execute the automated 5-step repair check
hoox repair check
```

The diagnostics engine runs a strict, sequential analysis of your entire monorepo environment:

hoox Repair Diagnostic Checklist

```
[Step 1] Submodule Integrity ..... ✓ COMPLETED
[Step 2] Dependency Resolutions ..... ✓ COMPLETED
[Step 3] TypeScript Type Safety ..... ✓ COMPLETED
[Step 4] Cloudflare Edge Bindings .... ✓ COMPLETED
[Step 5] Hardware Secrets Validation . ✓ COMPLETED
```

```
Diagnostic Result: 0 errors detected
```

Step Details

Step	Check	What It Validates
1	Submodule Integrity	All 9 worker folders under <code>workers/</code> are fully populated and registered in Git
2	Dependency Resolution	<code>node_modules</code> across Bun workspaces are complete with no missing or duplicate libraries
3	Type Safety	TypeScript compilation via <code>tsc --noEmit</code> catches syntax or interface errors
4	Edge Bindings	Connects to Cloudflare's API to verify D1, KV, Queues, R2, and Vectorize resources exist on your account
5	Secrets Validation	Audits your Workers Secrets to ensure all required exchange keys and tokens are securely bound



2. TARGETED COMPONENT REPAIRS

If a diagnostic check fails, you do not need to redeploy the entire stack. You can run highly targeted repairs:

```
# A. Re-verify, provision, and repair missing Cloudflare bindings
hoox repair infra

# B. Re-audit and sync encrypted Workers Secrets to Cloudflare
hoox repair secrets

# C. Reset the CONFIG_KV configuration namespace to default variables
hoox repair kv

# D. Re-apply Drizzle DQL schemas and run pending D1 database migrations
hoox repair db

# E. Rebuild and redeploy a single, degraded worker
hoox repair worker trade-worker

# F. Full interactive workspace rebuild (last resort)
hoox repair rebuild
```

Repair Command Quick Reference

Command	Use Case	Safe?	Downtime?
<code>hoox repair infra</code>	Missing D1/KV/Queue bindings on Cloudflare	✓ Yes	None
<code>hoox repair secrets</code>	Rotated API keys need re-binding	✓ Yes	None
<code>hoox repair kv</code>	Configuration drift, need factory defaults	⚠ Caution	<10s
<code>hoox repair db</code>	Schema mismatch or failed migrations	⚠ Caution	Seconds
<code>hoox repair worker</code> <code><name></code>	Single worker crash or build failure	✓ Yes	Seconds
<code>hoox repair rebuild</code>	Catastrophic failure, full system reset	🔴 Destructive	Minutes

⚡ 3. FULL SYSTEM INTERACTIVE REBUILD

In the event of a catastrophic system failure (e.g., lost Cloudflare credentials, corrupted SQLite files, or major monorepo drift), execute a full interactive self-healing rebuild:

```
# Execute an interactive, guided workspace rebuild
hoox repair rebuild
```

The Rebuild Sequence

When triggered, the CLI walks you through a structured recovery protocol:

Phase	Action	Automatic?
1	D1 Database Backup — Exports current ledger to <code>backups/recovery-pre-rebuild.sql</code>	✓ Yes
2	Infrastructure Tear-Down — Deletes corrupted D1, KV, and Queue instances	✓ Yes
3	Fresh Provisioning — Recreates all database tables and config buckets on Cloudflare	✓ Yes
4	Sequenced Redeployment — Re-compiles and deploys all 9 edge workers in correct dependency order	✓ Yes
5	Database Seeding — Restores backup SQL data and re-applies schema migrations	✓ Yes
6	KV Sync — Applies the 16-key configuration manifest defaults	✓ Yes

 Danger

The `hoox repair rebuild` command performs destructive resets on D1 and KV instances. Ensure you carefully read the interactive prompts and confirm database backup completion before letting the script proceed!

 4. COMMON TROUBLESHOOTING SCENARIOS

Symptom / Failure	Primary Root Cause	CLI Healing Action
<code>bun install</code> or build crashes	Git submodules are empty or out of sync	<code>git submodule update --init --recursive</code> then <code>hoox repair check</code>
<code>D1_ERROR: no such table</code>	SQLite D1 tables were not initialized	<code>hoox db apply --remote</code>
<code>500 Internal Server Error</code>	Missing or rotated exchange API secrets	<code>hoox secrets set BYBIT_API_KEY "key"</code> then <code>hoox deploy update-internal-urls</code>
Telegram Bot is mute	BotFather token is wrong, or webhook is unregistered	<code>hoox secrets set TELEGRAM_BOT_TOKEN "token"</code> then <code>hoox deploy telegram-webhook</code>
Queue depth growing indefinitely	Exchange API consistently returning errors	Check exchange API key validity; <code>hoox monitor queue-depth</code>
Dashboard at 500 errors	Next.js build failed or D1 schema outdated	<code>hoox repair db</code> then <code>hoox deploy dashboard</code>
All trades returning 409	Idempotency keys colliding from rapid duplicate webhooks	Verify TradingView <code>alert.freq_once_per_bar_close</code> setting; wait 24h for DO TTL cleanup

Emergency Decision Tree

```
Trading not working?
├─ Is the kill switch ON? → `hoox monitor kill-switch show`
│   ├── YES → `hoox monitor kill-switch off`
│   └─ NO ↓
├─ Are workers healthy? → `hoox monitor status`
│   ├── Any ✗ FAILED? → `hoox repair check`
│   └─ All ✓ OK ↓
├─ Are secrets bound? → `hoox secrets check`
│   ├── Any missing? → `hoox secrets set <NAME> <VALUE>`
│   └─ All present ↓
├─ Is queue depth > 0? → `hoox monitor queue-depth`
│   ├── YES → Exchange API issue; check exchange status page
│   └─ NO ↓
└─ Still stuck? → Open GitHub Issue with `hoox logs tail` output
```

Pro Tips

- **Scheduled Health Checks:** Set up a cron job to run `hoox repair check` daily and email yourself the results.
- **Pre-Deploy Checklist:** Always run `hoox repair check` and `hoox test` before deploying to production.
- **Secrets Rotation:** Rotate exchange API keys quarterly. Use `hoox secrets set` to update — no redeployment needed.

Next Steps

- **Monitoring Operations Guide** — Audit system status, tail console logs, and verify fills.
- **CLI Reference Manual** — Review all CLI commands, options, and JSON flags.
- **Configuration Dictionary** — Map all 31 environment variables and 16 KV manifest entries.

TRADINGVIEW WEBHOOK INTEGRATION

enduser/tutorials/tradingview-webhook.md



TRADINGVIEW WEBHOOK INTEGRATION

Connecting **TradingView Alerts** directly to your Hoox edge gateway is the most common way to automate your trading strategies. By combining TradingView's advanced charting engines with Hoox's low-latency edge execution, you can trigger orders instantly based on mathematical indicators, chart patterns, or custom Pine Script v5 logic.

This tutorial walks you through writing a Pine Script v5 script, setting up a TradingView webhook alert, and testing executions.



STEP 1: WRITING A PINE SCRIPT V5 SIGNAL INDICATOR

To trigger clean trade alerts, use TradingView's **Pine Script v5**. Below is a copy-paste indicator script that evaluates a Moving Average Cross strategy and generates standardized trade alert signals:

```
//@version=5
indicator("Hoox EMA Cross Signals", overlay=true)

// 1. Inputs & Parameters
fastLength = input.int(9, title="Fast EMA Length")
slowLength = input.int(21, title="Slow EMA Length")

// 2. Indicators Calculation
fastEMA = ta.ema(close, fastLength)
slowEMA = ta.ema(close, slowLength)

plot(fastEMA, color=color.blue, title="Fast EMA")
plot(slowEMA, color=color.orange, title="Slow EMA")

// 3. Trade Conditions
longCondition = ta.crossover(fastEMA, slowEMA)
shortCondition = ta.crossunder(fastEMA, slowEMA)

plotshape(longCondition, title="Long Cross", style=shape.triangleup, location=location.belowbar)
plotshape(shortCondition, title="Short Cross", style=shape.triangledown, location=location.abovebar)

// 4. Alert Payloads Construction (JSON compliant)
longAlertJson = '{"apiKey": "your-hoox-webhook-passkey", "exchange": "bybit", "action": "LONG", "type": "LONG"}'
shortAlertJson = '{"apiKey": "your-hoox-webhook-passkey", "exchange": "bybit", "action": "SHORT", "type": "SHORT"}'

// 5. Trigger Alerts
if (longCondition)
    alert(longAlertJson, alert.freq_once_per_bar_close)

if (shortCondition)
    alert(shortAlertJson, alert.freq_once_per_bar_close)
```

Tip

Replace `"your-hoox-webhook-passkey"` in your Pine Script with the actual passkey configured in your `CONFIG_KV` store (`webhooks:api_key`). This is a critical security step—the gateway will reject any alerts with mismatched keys with a `401 Unauthorized` error.



STEP 2: CREATING THE TRADINGVIEW WEBHOOK ALERT

Once your script is saved and added to your chart:

1. Click the **Alarm Clock (Alerts)** icon on the top right panel in TradingView.
2. Select **Condition**: Choose your indicator `"Hoox EMA Cross Signals"` and select `"Any Alert Function Call"` (this directs TradingView to parse the custom JSON payloads from the `alert()` functions inside your script).
3. Under **Expiration**: Select your alert lifespan (Premium accounts support Open-Ended alerts).
4. Navigate to the **Notifications** Tab:
 - Check the **Webhook URL** box.

- Paste your public Hoox Gateway endpoint URL: `https://hoox.alpha-trading.workers.dev/webhook`

5. Navigate to the **Settings** Tab:

- In the **Message** box, type: `{{strategy.order.alert_message}}` (if using a Backtesting Strategy) or leave it blank (if using an Indicator, as the JSON payload is already defined inside our `alert()` function).

6. Click **Create**.

STEP 3: VERIFYING EXECUTION IN PRODUCTION

When the Moving Average crossover occurs on your chart:

1. TradingView compiles the JSON payload and POSTs it to your edge gateway.
2. The `hoox` gateway validates the signature and routes the order to Bybit.
3. Check the transaction directly in your terminal:

```
hoox monitor trades
```

4. Stream live gateway telemetry logs to verify execution latency:

```
hoox logs tail hoox
```

WEBHOOK TROUBLESHOOTING RUNBOOK

Symptom / Error	Primary Cause	Resolution
Alert fires but no trade executes	Mismatched API keys or WAF block.	Run <code>hoox logs tail hoox</code> to stream traffic. Check if the client IP was dropped by Cloudflare WAF.
401 Unauthorized	Incorrect <code>apiKey</code> in Pine Script.	Run <code>hoox config kv get webhooks:api_key</code> to check your key. Paste the exact string into your Pine Script alert payload.
409 Conflict	Double-alert fired within the same bar.	Adjust the alert frequency in Pine Script: use <code>alert.freq_once_per_bar_close</code> instead of <code>alert.freq_all</code> to prevent rapid-fire retries.
Invalid Symbol API reject	Symbol format mismatch.	Hoox automatically converts variations (like <code>BTC-USDT</code> or <code>btc/usdt</code>) to <code>BTCUSDT</code> , but ensure your script sends standard uppercase symbols.

Next Steps

- **Setting Up Telegram Alerts** — Integrate Telegram notifications to get fills pushed directly to your phone.
- **API Endpoint Reference** — Review full request schemas and status codes.

TELEGRAM BOT SETUP

enduser/tutorials/telegram-bot.md

TELEGRAM BOT SETUP

The **telegram-worker** is a central operational pillar of the Hoox trading ecosystem. It serves two critical functions:

1. **Push Notifications:** Sends instant real-time order fill alerts, daily P&L audits, and system health status updates directly to your mobile phone.
2. **Interactive Command Console:** Provides a secure chat terminal allowing you to query open positions, check D1 balance logs, and trigger the Global Kill Switch directly via Telegram chat.

This tutorial guides you through creating a bot, binding secrets, deploying the secure Cloudflare webhook, and using commands.

STEP-BY-STEP CONFIGURATION PATH

Step 1: Provision a Bot via BotFather

1. Open the Telegram app and search for the verified **@BotFather** account.
2. Click **Start** and send the command:

```
/newbot
```

3. Enter a friendly name for your bot (e.g. **My Hoox Edge Bot**).
4. Choose a unique username ending in “bot” (e.g. **AlphaHooxTradeBot**).
5. BotFather will output your **HTTP API Token** (save this securely): **5829104829:AAFkL92jL-492kL1...**

Step 2: Retrieve Your Private Chat ID

To ensure that only **you** can command your bot (and to prevent unauthorized users from viewing your trading balances), you must capture your private Telegram Chat ID:

1. Search for the username **@userinfobot** in Telegram and start a chat.
2. The bot will instantly return your numeric **Id** (e.g., **987654321**).

Step 3: Inject Credentials via Hoox CLI

Bind your bot token and authorized Chat ID as encrypted Workers Secrets in your workspace:

```
# Inject the secure Telegram Bot token
hoox secrets set TELEGRAM_BOT_TOKEN "5829104829:AAFKL92jL-492kL1..."

# Inject your authorized Telegram Chat ID
hoox secrets set TELEGRAM_CHAT_ID "987654321"
```

Step 4: Register the Webhook with Telegram

To allow Telegram's servers to route incoming chat commands straight to your edge worker, you must register your deployed gateway URL as the bot's official webhook handler:

```
# Automatically register the webhook endpoint with Telegram's APIs
hoox deploy telegram-webhook
```

The CLI calls Telegram's API to configure the route: <https://hoox.alpha-trading.workers.dev/telegram-webhook>



INTERACTIVE CHAT COMMANDS

Open your private bot chat in Telegram, click **Start**, and send the following commands to manage your edge:

Command	Action	Description
<code>/start</code>	Onboarding	Verifies connection and returns system welcome message.
<code>/status</code>	System Audit	Probes all workers and returns online/offline health status.
<code>/trades</code>	Transaction Log	Retrieves a formatted table of the 5 most recent executed fills.
<code>/positions</code>	Exposure Audit	Lists all currently active long/short positions with unrealized P&L.
<code>/kill_on</code>	Emergency Halt	Instantly activates the Global Kill Switch in KV, halting all trading.
<code>/kill_off</code>	Resume	Deactivates the Global Kill Switch, restoring signal processing.

CONVERSATIONAL AI CHAT & CHART AUDITS

Beyond static commands, `telegram-worker` is integrated with **Cloudflare Workers AI** and the **Multi-Provider AI Gateway**:

- **Natural Language Queries:** You can ask the bot conversational questions like “*Should I close my BTC position?*” or “*Analyze my trade win rate today*”. The bot queries your D1 SQL database, aggregates context using Vectorize, and responds with a LLaMA-3 analytical summary.
- **Chart Image Audits:** If you upload a screenshot of a trading chart or a portfolio page, the AI Gateway uses vision models (like Claude 3.5 Sonnet or Gemini 1.5 Pro) to analyze the chart and return an automated risk audit.

Warning

The `telegram-worker` strictly filters all incoming messages. If a message originates from a `Chat ID` that does not match your authorized `TELEGRAM_CHAT_ID` secret, it is silently dropped and logged as a security alert, ensuring your account remains 100% secure.

Next Steps

|- **Alert Troubleshooting** — Common TradingView webhook errors and resolutions. |- **Platform Configuration Guide** — Comprehensive dictionary of all 31 env keys and 16 KV key-value items.

EMAIL SIGNAL ROUTING

enduser/tutorials/email-signals.md



EMAIL SIGNAL ROUTING

The **email-worker** is an ancillary input plugin that allows you to trigger automated trades via raw email alerts. While webhooks are the standard path, many legacy systems, charting platforms, or custom newsletters only distribute trade signals via email.

This tutorial walks you through setting up **email-worker** credentials, configuring regex subject scanning patterns, and securely routing messages.



1. INJECTING EMAIL CONNECTION CREDENTIALS

To allow **email-worker** to log into your dedicated signals inbox, inject your SMTP/IMAP mailbox credentials as encrypted Workers Secrets:

```
# 1. Inject the IMAP/POP3 host address
hoox secrets set EMAIL_HOST "imap.securemail.com"

# 2. Inject the mailbox username/email address
hoox secrets set EMAIL_USER "signals@my-trading-app.com"

# 3. Inject the secure mailbox app password
hoox secrets set EMAIL_PASS "your_app_password_here"
```

⚠ Warning

Always use a **dedicated** email address solely for trade signals. Never use your personal inbox. Additionally, configure your email provider (e.g., Gmail, Fastmail) to require an **App Password** rather than your master account credentials to ensure tight access control.



2. CONFIGURING REGEX PARSING RULES IN KV

Once **email-worker** establishes connection, it scans the inbox on a background schedule, evaluating incoming subjects and message bodies against **Regular Expression (regex)** patterns defined in **CONFIG_KV** :

```
# A. Define the prefix pattern required in the email subject line
hoox config kv set email:scan_subject "^TRADE SIGNAL:.*"

# B. Define the regex pattern used to extract the target asset token
hoox config kv set email:coin_pattern "(BTC|ETH|SOL|LINK|AVAX)"

# C. Define the regex pattern used to resolve trade action
hoox config kv set email:action_pattern "(LONG|SHORT|CLOSE|BUY|SELL)"
```

The Parsing Mechanism

When an email is scanned:

1. The worker matches the subject. If the subject is `"TRADE SIGNAL: Breakout Detected"`, the check passes.
2. The worker scans the email body using the regex patterns:
 - Body: `"...we are entering a LONG position on SOL/USDT..."`
 - Hitting `email:coin_pattern` resolves: `SOL`
 - Hitting `email:action_pattern` resolves: `LONG` (translated internally to `BUY`).
3. Automatically triggers an order via the `trade-worker` service binding.



3. SECURITY AUDITS: SPF & DKIM VALIDATION

To prevent malicious “spoofing” (e.g., an unauthorized sender trying to forge a trade signal to your inbox):

- **Sender Validation:** The `email-worker` parses the email’s headers and matches the sender’s address against a safe allowlist in KV: `email:authorized_senders = ["alerts@tradingview.com"]`.
- **DNS Verification:** The worker validates the **SPF (Sender Policy Framework)** and **DKIM (DomainKeys Identified Mail)** headers to verify that the message physically originated from the authorized domain and was not intercepted.



4. TESTING YOUR EMAIL PIPELINE

To verify that the email signal ingestion loop works perfectly:

1. Compose a mock email from your authorized sender address.
2. Subject: `TRADE SIGNAL: Cross Over`
3. Body: `Action: SHORT, Symbol: ETHUSDT, Quantity: 0.05`
4. Send to your dedicated inbox.

5. In your terminal, tail the email worker's runtime logs to confirm the parse and order submission:

```
hoox monitor logs email-worker
```

Next Steps

- | - **Alert Troubleshooting** — Common TradingView webhook errors and resolutions. | - **Platform Configuration Guide** — Comprehensive dictionary of all 31 env keys and 16 KV key-value items.
- **API Endpoint Reference** — Review full REST payloads and HTTP error returns.

CLI COMMANDS REFERENCE

enduser/reference/cli-commands.md



CLI COMMANDS REFERENCE

The `@jango-blockchained/hoox-cli` tool manages the entire monorepo development, provisioning, deployment, monitoring, and self-healing pipelines. This reference provides the complete command tree, positional arguments, optional flags, and concrete examples for all 15 command groups.



GLOBAL FLAGS

These options are registered globally and can be appended to any command:

Flag	Description	Example
<code>--json</code>	Outputs machine-parseable JSON format (ideal for script pipelines).	<code>hoox monitor status --json</code>
<code>--quiet</code>	Suppresses headers, banners, and interactive prompts.	<code>hoox deploy kv-config --quiet</code>
<code>--help</code>	Prints complete parameter and option listings.	<code>hoox infra d1 --help</code>
<code>--version</code>	Outputs active CLI package build version.	<code>hoox --version</code>



COMMAND GROUPS DIRECTORY

hoox	Launch TUI (when run with no args)
└─ init	Interactive multi-phase setup wizard
└─ clone [name]	Bootstrap workspace from template
└─ dev	Local development execution engine
└─ start	Start all workers (native or Docker)
└─ worker <name>	Start a single worker
└─ dashboard	Start Next.js dashboard dev server
└─ deploy	Edge deployment pipelines
└─ all	Roll out all workers + dashboard in sequence
└─ worker <name>	Deploy a single edge worker
└─ dashboard	Deploy Next.js dashboard via OpenNext
└─ telegram-webhook	Register Telegram bot webhook API
└─ update-internal-urls	Bind inter-worker Service Binding URLs
└─ kv-config	Synchronize KV manifest variables
└─ infra	Cloudflare resource provisioning IaC
└─ provision	Auto-provision all required services
└─ d1 [list/create/delete]	Manage D1 databases
└─ kv [list/create/delete]	Manage KV namespaces
└─ r2 [list/create/delete]	Manage R2 object storage buckets
└─ queues [list/create/delete]	Manage Cloudflare Queues
└─ vectorize [create/delete]	Manage vector search indexes
└─ config	Workspace and environmental variables
└─ env [init/show/validate]	Manage .env.local build variables
└─ kv [set/get/list/delete]	Get/Set dynamic KV runtime parameters
└─ show/set	View or set wrangler.jsonc values
└─ check	Toolchain and route diagnostic suite
└─ prerequisites	Validate local machine tool installs
└─ setup	Audit .env configurations
└─ health	Probe worker /health endpoints
└─ db	SQLite D1 Database administration
└─ apply	Apply DDL schemas to D1
└─ migrate	Run schema migrations
└─ list	Display active SQLite tables
└─ query <sql>	Execute custom SQL reads
└─ export	Dump database as a secure SQL file
└─ reset	Destructive truncate of all tables
└─ monitor	Observability and emergency controls
└─ status	Probe active worker health routes
└─ trades [N]	Aggregate recent filled transactions
└─ logs [worker]	Tail and inspect time-series logs
└─ queue-depth	Audit message counts in queues
└─ backup	Backup SQL ledger to backups/
└─ kill-switch [show/on/off]	Emergency global trading halt
└─ repair	System diagnostics & self-healing
└─ check	Run 5-step checklist
└─ worker <name>	Rebuild a degraded worker
└─ infra	Verify and repair missing edge bindings
└─ secrets	Re-sync hardware secrets
└─ kv	Restore KV manifest defaults
└─ db	Re-apply Drizzle migrations
└─ rebuild	Full destructive interactive rebuild
└─ logs	Time-series log extraction
└─ tail <worker>	Stream live edge logging console
└─ download <worker>	Download logs to a local file

```
| test
| waf
```

Run verification pipeline (CI)
Auto-configure Cloudflare WAF firewall rules

KEY COMMANDS DETAIL & EXAMPLES

A. Initialization & Bootstrap

```
# Onboard a new machine, configure accounts and select worker profile
hoox init

# Verify that git, bun, wrangler, and docker are correctly configured
hoox check prerequisites
```

B. Local Development

```
# Start all workers in a Docker container stack
hoox dev start --runtime docker

# Start only the Next.js frontend developer server
hoox dev dashboard
```

C. Edge Provisioning & Deployment

```
# Provision all databases, buckets, and queues on your Cloudflare account
hoox infra provision

# Deploy the entire microservices stack in sequence, automatically updating URLs
hoox deploy all --auto
```

D. Operational Monitoring & Emergency halting

```
# Emergency Halt: halt all trade signals globally in under 10 seconds
hoox config kv set trade:kill_switch true

# Audit active trade history table
hoox monitor trades 25
```

E. Database Administration

```
# Query the total sum of fees paid today across Bybit
hoox db query "SELECT SUM(fee) FROM trades WHERE created_at >= date('now')" --remote
```

F. Self-Healing & Diagnostics

```
# Run a full workspace system audit  
hoox repair check
```

Tip

Every single subcommand is fully documented locally! Append `--help` to any command (e.g. `hoox config env --help`) to view advanced positional arguments and specific flag options instantly.

Next Steps

|- **Configuration Dictionary** — Comprehensive dictionary of all 31 env keys and 16 KV key-value items.

- **API Endpoint Reference** — Analyze REST routes, schemas, and gateway error structures.

API ENDPOINT SPECIFICATION

enduser/reference/api-endpoints.md



API ENDPOINT SPECIFICATION

This document details the public REST API endpoints exposed by the Hoox gateway (`workers/hoox`). These endpoints are used to ingest automated trade signals, register Telegram bots, query AI metrics, and check serverless V8 isolate health status.



REQUEST HEADERS & SECURITY POLICY

Every public HTTP request submitted to the Hoox gateway must include the following headers:

```
Content-Type: application/json
Accept: application/json
```

CORS & Origin Policies

For security, **Cross-Origin Resource Sharing (CORS)** is disabled by default on the `/webhook` route—it only accepts direct server-to-server POST requests (e.g. from TradingView or custom server scripts).

To interact with the dashboard, the gateway verifies active authorization cookies and tokens inside the V8 engine context.



1. INGEST TRADE SIGNAL WEBHOOK

Receives, validates, and routes trade orders to exchange APIs.

- **Endpoint:** `/webhook`
- **Method:** `POST`

Request Payload (JSON Schema)

```
{
  "apiKey": "alpha_secure_key_18305",
  "exchange": "bybit",
  "action": "LONG",
  "symbol": "BTCUSDT",
  "quantity": 0.005,
  "leverage": 10,
  "idempotencyKey": "uuid-9b1deb4d-3b7d"
}
```

Response Models

A. Success Path (Order Filled Instantly - 200 OK)

```
{
  "success": true,
  "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "exchange": "bybit",
  "symbol": "BTCUSDT",
  "action": "LONG",
  "result": {
    "orderId": "18049284739",
    "status": "Filled",
    "executedQty": 0.005,
    "price": 68425.5,
    "timestamp": 1779261050000
  }
}
```

B. Queue Failover Path (Exchange Offline / Rate-limited - 202 Accepted)

If the exchange is down, the signal is enqueued for guaranteed asynchronous delivery.

```
{
  "success": true,
  "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "status": "Enqueued",
  "message": "Signal enqueued in trade-execution Queue for background execution retry."
}
```

2. SYSTEM HEALTH CHECK

Probes the gateway isolate, validating connections to D1 databases and CONFIG_KV caches.

- **Endpoint:** `/health`
- **Method:** `GET`

Success Response (200 OK)

```
{
  "status": "ok",
  "timestamp": 1779261050000,
  "bindings": {
    "d1": "connected",
    "kv": "connected",
    "queue": "active"
  }
}
```



3. CONVERSATIONAL AI CHAT & TELEMETRY

Integrates with the `agent-worker` Multi-Provider AI Gateway.

A. Conversational Chat Stream

- Endpoint: `/agent/chat`
- Method: `POST`
- Headers: `Accept: text/event-stream` (Supports Server-Sent Events)

Request Payload

```
{
  "prompt": "Evaluate my trade performance over the last 24 hours.",
  "stream": true,
  "provider": "anthropic"
}
```

B. AI Gateway Telemetry

- Endpoint: `/agent/usage`
- Method: `GET`

Response Payload (200 OK)

```
{
  "success": true,
  "timestamp": 1779261050000,
  "usage": {
    "openai": { "inputTokens": 45180, "outputTokens": 8920, "costUSD": 0.183 },
    "anthropic": {
      "inputTokens": 18240,
      "outputTokens": 4010,
      "costUSD": 0.081
    },
    "workersAi": { "neuronsUsed": 842000, "costUSD": 0.0 }
  }
}
```



4. STANDARD PLATFORM ERROR MODELS

When an API check fails, the gateway uses the shared `@jango-blockchained/hoox-shared` package to return a standardized JSON error format:

A. 400 Bad Request (Payload Validation Failure)

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid JSON payload structure.",
    "details": [
      { "field": "quantity", "issue": "Must be greater than zero." },
      {
        "field": "exchange",
        "issue": "Invalid exchange router. Options: binance, bybit, mexc."
      }
    ]
  }
}
```

B. 401 Unauthorized (Invalid API Key / IP Address)

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication failed. Mismatched apiKey or unauthorized origin IP."
  }
}
```

C. 409 Conflict (Duplicate Request Intercepted)

```
{
  "success": false,
  "error": {
    "code": "DUPLICATE_REQUEST",
    "message": "Order rejected. Durable Object locked: trace ID already processed in the last 24 hours."
  }
}
```

D. 503 Service Unavailable (Kill Switch Engaged)

```
{
  "success": false,
  "error": {
    "code": "KILL_SWITCH_ACTIVE",
    "message": "Trading halted globally. The emergency Global Kill Switch is currently turned ON."
  }
}
```

Tip

Every error code is designed to map directly to a readable prompt in the Telegram bot and Next.js dashboard UI, allowing you to instantly diagnose execution problems.

Next Steps

|- **Request Payload Schemas** — Analyze the complete JSON request schemas and TypeScript interfaces.

- **CLI Commands Reference** — Review all CLI commands, options, and JSON flags.

CONFIGURATION DICTIONARY

enduser/reference/configuration.md



CONFIGURATION DICTIONARY

This document serves as the absolute, exhaustive reference dictionary for every environment variable, encrypted Workers Secret, and dynamic KV runtime setting utilized in the Hoox trading ecosystem.

1. ENVIRONMENT VARIABLES & SECRETS MATRIX (31 KEYS)

These parameters are configured in your local `.env.local` file for building/deploying or injected as encrypted **Workers Secrets** for runtime isolate compute.

A. Core Platform & Infrastructure

Variable	Required	Type	Default	Description
<code>CLOUDFLARE_API_TOKEN</code>	Yes	<code>string</code>	—	Cloudflare API Token with Workers, D1, and KV read/write permissions.
<code>CLOUDFLARE_ACCOUNT_ID</code>	Yes	<code>string</code>	—	Your 32-character Cloudflare Dashboard Account hash.
<code>SUBDOMAIN_PREFIX</code>	Yes	<code>string</code>	—	Subdomain prefix under which public worker gateways deploy.
<code>NODE_ENV</code>	No	<code>string</code>	<code>production</code>	Environment profile: <code>development</code> , <code>staging</code> , or <code>production</code> .
<code>HOOX_API_URL</code>	No	<code>string</code>	—	Local API target URL (automatically configured during dev runs).

B. Exchange API Credentials (Encrypted Secrets)

Variable	Required	Type	Scope	Description
<code>BYBIT_API_KEY</code>	No	string	trade-worker	Bybit order placement account credential.
<code>BYBIT_API_SECRET</code>	No	string	trade-worker	Bybit HMAC-SHA256 private order signature.
<code>BINANCE_API_KEY</code>	No	string	trade-worker	Binance trade permission credential.
<code>BINANCE_API_SECRET</code>	No	string	trade-worker	Binance private HMAC order signature.
<code>MEXC_API_KEY</code>	No	string	trade-worker	MEXC trade permission credential.
<code>MEXC_API_SECRET</code>	No	string	trade-worker	MEXC private HMAC order signature.

C. Telegram Bot Alerts & Telemetry

Variable	Required	Type	Scope	Description
<code>TELEGRAM_BOT_TOKEN</code>	No	string	telegram-worker	Telegram HTTP API bot auth token from <code>@BotFather</code> .
<code>TELEGRAM_CHAT_ID</code>	No	string	telegram-worker	Authorized numeric Chat ID for pushed fills and commands.

D. Multi-Provider AI Credentials

Variable	Required	Type	Scope	Description
<code>OPENAI_API_KEY</code>	No	string	agent-worker	OpenAI API access key.
<code>ANTHROPIC_API_KEY</code>	No	string	agent-worker	Anthropic Claude API access key.
<code>GOOGLE_AI_API_KEY</code>	No	string	agent-worker	Google Gemini API access key.

E. DeFi & Web3 Wallet Settings

Variable	Required	Type	Scope	Description
<code>ETH_MNEMONIC</code>	No	string	web3-wallet	Secure 12 or 24-word seed phrase for on-chain contract swaps.
<code>RPC_PROVIDER_URL</code>	No	string	web3-wallet	HTTP Ethereum / EVM JSON-RPC provider (e.g. Infura/Alchemy).

F. Email Parsing Inbox Connection

Variable	Required	Type	Scope	Description
<code>EMAIL_HOST</code>	No	string	email-worker	POP3/IMAP mailbox server address.
<code>EMAIL_USER</code>	No	string	email-worker	Target signal mailbox email address.
<code>EMAIL_PASS</code>	No	string	email-worker	Secure app password for signal mailbox access.

2. DYNAMIC KV MANIFEST SETTINGS (16 KEYS)

These runtime variables exist inside the `CONFIG_KV` namespace. They are accessed globally at sub-millisecond speeds and take effect instantly upon being modified.

KV Key	Type	Default	Operational Impact
<code>trade:kill_switch</code>	boolean	false	Global emergency halt. If <code>true</code> , all executions halt.
<code>trade:max_daily_drawdown_percent</code>	number	5.0	Maximum daily loss before the AI Risk Manager triggers the kill switch.
<code>webhooks:api_key</code>	string	your_key	Public webhook passkey checked during signal ingestion.
<code>webhooks:queue_mode</code>	string	queue_failover	Routing behavior: <code>direct_only</code> , <code>queue_failover</code> , <code>queue_everywhere</code> .
<code>exchanges:default_routing</code>	string	bybit	Default CEX fallback router: <code>binance</code> , <code>bybit</code> , or <code>mexc</code> .
<code>routing:dynamic:enabled</code>	boolean	false	Enables/disables dynamically shifting routes based on symbols.
<code>email:scan_subject</code>	string	TRADE	Prefix required in signal email subjects.
<code>email:coin_pattern</code>	string	(BTC ETH)	Regex expression used to extract asset tokens from email bodies.
<code>email:action_pattern</code>	string	(BUY SELL)	Regex expression used to resolve buy/sell actions in email bodies.
<code>email:authorized_senders</code>	array	[]	List of email addresses authorized to submit trade signals.
<code>webhook:tradingview:ip_check</code>	boolean	false	Toggles dropping payloads that don't originate from TradingView IPs.

 Tip

You can sync, check, or reset this entire KV manifest in one command: `hoox config kv apply-manifest !`

 Next Steps

- **5-Minute Quick Start Guide** — Deploy all workers and fire your first simulated webhook.
- **API Endpoint Reference** — Review full REST schemas, parameters, and error models.